# Polytime Reductions

*Some examples.*

## Contents


**1 About Running Times and Space Requirements** **1**

**2 A Useful 3SAT Lemma** **2**

**3 Some NP-Complete Languages** **4**

3.1 X1-3SAT . . . . . 4
3.2 NAE-3SAT . . . . . 6
3.3 CLIQUE . . . . . 8
3.4 IS . . . . . 11
3.5 VC . . . . . 13
3.6 3COL . . . . . 14
3.7 HAM-PATH . . . . . 19
3.8 HAM-CYCLE . . . . . 24
3.9 INDUCED-PATH . . . . . 25
3.10 PARTITION-INTO-TRIANGLES . . . . . 29
3.11 3DM . . . . . 32
3.12 SUBSET-SUM . . . . . 35
3.13 JOB-SEQUENCING . . . . . 38
3.14 TIMETABLE-SCHEDULING . . . . . 42
3.15 CUBIC-SUBGRAPH . . . . . 50
3.16 NONOGRAM . . . . . 58


## 1 About Running Times and Space Requirements

As I've said in class, we'll sometimes ignore a logarithmic factor in our running time/space analysis when said logarithmic factor doesn't have any bearing on whether an algorithm is polytime or not. See the week 6 slides for more details.

Specifically, we'll often ignore the logarithmic factors that show up when we're using pointers or indices. Because of this, we'll often say, for example, that $\langle G = (V, E)\rangle$ has a size of $\mathcal{O}(|V| + |E|)$ or of $\mathcal{O}(|V|^2)$ rather than $\mathcal{O}([|V| + |E|] \log |V|)$ or of $\mathcal{O}(|V|^2 \log |V|)$.

There are a couple of reasons for doing this:

1. It simplifies our analysis.

2. It brings our running times in line with what you may be used to in, say CSCB63.

   *In B63 we say that a BFS on a graph $G = (V, E)$, for example, takes $\mathcal{O}(|V| + |E|)$ time. But we actually need $\mathcal{O}(\log |V|)$ space and time just to read the indices for the vertices in G! So it's really closer to $\mathcal{O}([|V| + |E|] \log |V|)$. There's a semi-hidden bound on input sizes in B63 that makes this reasonable. But we'll continue to call it $\mathcal{O}(|V| + |E|)$ here for continuity's sake.*

3. Finally, In the current section of the course we're primarily interested in whether a program runs in polynomial time. The logarithmic factors that we're ignoring never change the polytime/non-polytime status of a program.

While the choice to ignore these factors makes the overall program analysis simpler, you should be aware that we're making it, and you should be prepared to include logarithmic factors when they are necessary for a proper analysis:

- If a logarithm is important to the question as to whether an algorithm runs in polynomial time, you shoukd include it.

  In particular, if a number $n$ is included as part of an input, it will usually be written in binary or decimal notation. So the size of this input will be $\mathcal{O}(\log n)$. So you have to be careful about looping over all $i$ from 1 to $n$.

- If you are running a logspace analysis, you won't be asking whether something uses polynomial amount of space, but rather a logarithmic amouont of space. You can't ignore logarithmic factors in this situation.

- If you are working to give a very tight complexity bound, or need to use a counting argument to show things are working the way you expect, you'll need to account for logarithmic factors.

- If you are working to give an optimized algorithm, you'll care about more than just making it run in polytime, and so logarithmic factors will matter. So a course like CSCC73 may well require you to be more exact about your running times. Be prepared for this.

# 2 A Useful 3SAT Lemma

In the following examples we'll often reduce from 3SAT as a basis, and when we do it will be useful to argue that each clause of a given 3CNF formula $\phi$ has three distinct variables as its literals. We formalize this idea here.

**Lemma:**

Suppose that we are given a 3SAT formula $\phi$. Then we can efficiently find an equivalent 3SAT formula $\phi'$ such that:

*i*) $\phi'$ uses the same variables as $\phi$, with the possibility of a small number of extra variables,

*ii*) For each satisfying truth assignment $\tau$ of $\phi$ there is exactly one equivalent truth assignment $\tau'$ of $\phi'$, and

*iii*) This $\tau'$ differs from $\tau$ only in its added variables.

*iv*) $\phi'$ has three distinct variables in each clause.

**Proof:**

Suppose we are given an arbitrary 3SAT formula $\phi$. Now, if $\phi$ has three distinct variables in each clause, then we are done: just set $\phi' = \phi$. So suppose that $\phi$ has clauses with repeated variables.

Then it either has clauses with a variable and its negation $((x \vee \neg x \vee y))$ or it has a clause with a literal that occurs twice.

We will firstly introduce the three variables $a, b$, and $c$, where $a, b$, and $c$ are distinct from the old variables. We then add the clauses

$$(\neg a \vee \neg b \vee \neg c) \wedge (\neg a \vee \neg b \vee c) \wedge (\neg a \vee b \vee \neg c) \wedge (\neg a \vee b \vee c) \wedge (a \vee \neg b \vee \neg c) \wedge (a \vee \neg b \vee c) \wedge (a \vee b \vee \neg c)$$

to the formula. You can see that the only way to assign values to $a, b$, and $c$ to make these clauses $T$ is $a = b = c = F$.

Now, a clause of the form $(x \vee \neg x \vee y)$ will always be satisfied, and so if we replace it with the clause $(x \vee \neg a \vee y)$, we will have another clause that will always be satisfied (we could also remove the clause altogether, but that would risk removing the variable $x$ from the formula). If we repeat this process until all such clauses (i.e., clauses that contain both some variable $x_i$ and its negation $\neg x_i$), we will have a new $\phi'$. If any clause of $\phi'$ still contains repeated variables, it must contain a repeated literal.

If there are clauses of this type, we can replace it as follows: Whenever we see a clause of the form $(x \vee x \vee y)$, we can replace it with $(x \vee a \vee y)$. Similarly, we replace clauses of the form $(x \vee x \vee x)$ with the clause $(x \vee a \vee b)$.

If $\phi'$ is the formula we obtain in this fashion, we can see:

i) $\phi'$ uses the same variables as $\phi$, with the possibility of new added variables ($a$, $b$, and $c$).

*ii)* For each satisfying truth assignment $\tau$ of $\phi$ there is exactly one equivalent truth assignment $\tau'$ of $\phi'$. The truth assignment $\tau'$ is just $\tau$ with the added assignment $a = b = c = F$.

*iii)* As we just said in point *ii)*, $\tau'$ differs from $\tau$ only in its added variables.

*iv)* Finally, $\phi'$ contains no clause with repeated variables.

So if I preprocess any $\phi$ I see with this strategy, I'll always get three distinct variables per clause.

In fact, we can also see that $\phi'$ contains at most seven more clauses then $\phi$, and that all of its clauses can be efficiently found using $\phi$. So this entire process can be run in polynomial time.

■

# 3 Some NP-Complete Languages

In the following examples we'll assume that the 3SAT and SAT languages are NP-complete.

---

## 3.1 X1-3SAT

Definition of X1-3SAT (*Exactly-One-3SAT*):

**INPUT:** A 3CNF $\phi$.

**QUESTION:** Does $\phi$ have a satisfying assignment in which exactly one literal in each clause is set to true?

**Reduced from:** 3SAT.

Firstly, we can see that this is in NP:

1) It is a yes/no problem.
2) Its certificate is simply a truth assignment $\tau$ to the variables in $\phi$.
3) The size of this certificate is $\mathcal{O}(k)$, where $k$ is the number of variables in $\phi$. Since every variable occurs at least one in the clauses, this certificate is polynomial in the size of $\phi$.
4) A verifier for this certificate would be:

   Let $\mathcal{V}$ =“On $\langle \phi, \tau \rangle$:”

   1. For $i = 1$ to $n$ (where $n$ is the number of clauses in $\phi$):
      $\mathcal{O}(n)$ *iterations.*
   2. Use $\tau$ to assign truth values to the literals in the clauses of the clause $c_i$.
      $\mathcal{O}(n)$ *time per iteration.*
   3. If $\tau$ does not set exactly one literal in $c_i$ to true, *reject.*
      $\mathcal{O}(1)$ *time per iteration.*
   4. *Accept.*
      $\mathcal{O}(1)$ *time.*
5) We can see that this verifier runs in $\mathcal{O}(n^2)$ time, and so is polytime in the size of $\phi$.

So X1-3SAT is in NP.

**The reduction** 3SAT $\leqslant_p$ X1-3SAT**:**

Let an input 3CNF $\phi$ be given. Note that we can assume using the lemma in section 2 that the clauses in $\phi$ each have three distinct variables.
We will build a new $\phi'$ as follows: For each clause $C_i = (\ell_{i_1} \vee \ell_{i_2} \vee \ell_{i_3})$, we introduce the six new variables $a_i$, $b_i$, $c_i$, $d_i$, and $e_i$. We replace the clause $C_i$ with the new clauses

- $(a_i \vee b_i \vee c_i)$,
- $(a_i \vee \neg \ell_{i_1} \vee d_i)$,

- $(b_i \vee \neg \ell_{i_2} \vee e_i)$,
- $(c_i \vee \neg \ell_{i_3} \vee f_i)$,

The idea behind these clauses is as follows: in the X1-3SAT problem, we have to be able to find one true literal per clause, while the other two literals must be false. We'll use this property to extend any satisfying truth assignment $\tau$ for $\phi$ into a satisfying assignment (satisfying in the X1-3SAT sense) $\tau'$ for $\phi'$. If $\tau$ is not satisfying, no such $\tau'$ will be possible. Given a satisfying $\tau$, then, $\tau'$ will encode a guess as to which of the three literals in $C_i$ is true $\tau$, and then check that guess (more than one can be true, of course).

The first of these clauses, the $(a_i \vee b_i \vee c_i)$, acts as a choice for which of the three literals $\ell_{i_1}$, $\ell_{i_2}$, and $\ell_{i_3}$, we expect to be true. So if we set $a_i = T$, we're asserting that $\ell_{i_1}$ is also $T$ under $\tau$, while $\ell_{i_2}$ and $\ell_{i_3}$ may take either value.

Clearly, if we make a good guess (if we can choose a true literal in $C_i$ under $\tau$), then $C_i$ is satisfied by $\tau$. And if $C_i$ is satisfied by $\tau$, we can make a good guess.

> If we set, say, $a_i = T$ and $\ell_{i_1} = T$ under $\tau$ as well, then the first clause, then our second new clause $(a_i \vee \neg \ell_{i_1} \vee d_i)$ evaluates to $(T \vee F \vee d_i)$, where $d_i$ can take either value. Setting $d_i$ to $F$ will allow us to set exactly one literal in this clause to $T$.
>
> Similarly, the clauses $(b_i \vee \neg \ell_{i_2} \vee e_i)$ and $(c_i \vee \neg \ell_{i_3} \vee f_i)$ will set to $(F \vee \neg \ell_{i_2} \vee e_i)$ and $(F \vee \neg \ell_{i_3} \vee f_i)$, respectively. Setting $e_i = \ell_{i_2}$ and $f_i = \ell_{i_3}$ lets us assign exactly one literal per clause to true for all of our clauses.
>
> The analysis is similar for the choices $b_i = T$ and $c_i = T$.
>
> So if $\tau$ is a satisfying assignment (and so $C_i$ is satisfied by $\tau$), we can extend $\tau$ to a certificate to $\phi'$ that satisfies the clause $C_i$.

On the other hand, if we make a bad guess the truth assignment should not extend (in particular, if there is no true literal in $C_i$ then we cannot make a good guess.)

> If we set, say, $a_i = T$ and $\ell_{i_1} = F$, then the clause $(a_i \vee \neg \ell_{i_1} \vee d_i)$ evaluates to $(T \vee T \vee d_i)$. This truth assignment will not satisfy the X1-3SAT criteria, regardless of the other variable assignments.
>
> The analysis for the choices $b_i = T$ and $c_i = T$ in this case, as well.

So if we repeat this procedure for every clause, we find:

> $\Rightarrow$) If $\phi$ has a satisfying truth assignment $\tau$, then we can extend $\tau'$ to the new variables as described above to get a certificate that $\langle \phi' \rangle \in$ X1-3SAT.
>
> $\Leftarrow$) If $\phi'$ has a certificate $\tau'$ showing it is in X1-3SAT, then we can restrict $\tau'$ to the variables in $\phi$ to get a truth assignment $\tau$. By our arguments above, $\tau$ satisfies $\phi$, so that $\langle \phi \rangle \in$ 3SAT.

Finally, we can see that this reduction replaces each of the $n$ clauses of $\phi$ with four new clauses, so that the result is a 3CNF of size $4n$. Each of these clauses can be found in polytime, and so the entire construction is polytime, as required.

Therefore, X1-3SAT is NP-complete.

■

---

## 3.2 NAE-3SAT

Definition of NAE-3SAT (*Not-All-Equal-3SAT*):

**INPUT:** A 3CNF $\phi$.

**QUESTION:** Does $\phi$ have a satisfying assignment in which at least one literal in each clause is set to true at least one literal in each clause is set to false?

**Reduced from:** 3SAT.

Firstly, we can see that this is in NP:

1) It is a yes/no problem.
2) Its certificate is simply a truth assignment $\tau$ to the variables in $\phi$.
3) The size of this certificate is $\mathcal{O}(k)$, where $k$ is the number of variables in $\phi$. Since every variable occurs at least one in the clauses, this certificate is polynomial in the size of $\phi$.
4) A verifier for this certificate would be:

   Let $\mathcal{V}$ ="On $\langle \phi, \tau \rangle$:"

   1. For $i = 1$ to $n$ (where $n$ is the number of clauses in $\phi$):
      $\mathcal{O}(n)$ *iterations.*
   2. Use $\tau$ to assign truth values to the literals in the clauses of the clause $c_i$.
      $\mathcal{O}(n)$ *time per iteration.*
   3. If $\tau$ does not set either exactly one or exactly two literals to true in $c_i$ to true, *reject.*
      $\mathcal{O}(1)$ *time per iteration.*
   4. *Accept.*
      $\mathcal{O}(1)$ *time.*
5) We can see that this verifier runs in $\mathcal{O}(n^2)$ time, and so is polytime in the size of $\phi$.

So NAE-3SAT is in NP.

**The reduction** 3SAT $\leqslant_p$ NAE-3SAT**:**

Let an input 3CNF $\phi$ be given. We will build the formula $\phi'$ as follows:

- The variables in $\phi'$ will be the variables $x_i$ in $\phi$, plus an extra variable $x_T$ not in $\phi$. We will also add three variables per clause $C_j$: $a_j$, $b_j$, and $c_j$.
- For each clause $C_j = (\ell_{j,1} \vee \ell_{j,2} \vee \ell_{j,3})$ in $\phi$ we will add the following clauses to $\phi'$:

  $(\neg x_T \vee \ell_{j,1} \vee a_j)$,

  $(\neg a_j \vee \ell_{j,2} \vee b_j)$,

  $(\neg b_j \vee \ell_{j,3} \vee c_j)$, and

  $(c_j \vee x_T \vee x_T)$.

*Note that we do not include the clause* $(\ell_{j,1} \vee \ell_{j,2} \vee \ell_{j,3})$ *in* $\phi'$.

Once we've done this for every clause, we return the resulting $\phi'$.

We argue that $\langle\phi\rangle \in$3SAT iff $\langle\phi'\rangle \in$NAE-3SAT.

**Proof that** $\langle\phi\rangle \in$3SAT **iff** $\langle\phi'\rangle \in$NAE-3SAT**:**

**Suppose that** $\langle\phi\rangle \in$3SAT**:**

Then it has a satisfying truth assignment $\tau$.

We create the new truth assignment $\tau'$ by extending $\tau$ to the new variables. After copying the variable assignments from $\tau$ to $\tau'$, we set $x_T = T$.

Given any clause $C_j$, we set the values of $a_j$, $b_j$, and $c_j$ as follows:

- If $\ell_{j,1} = T$, we set $a_j = F$. Otherwise we set $a_j = T$.
- If $\ell_{j,2} = T$ or $a_j = F$, we set $b_j = F$. Otherwise we set $b_j = T$.
- If $\ell_{j,3} = T$ or $b_j = F$, we set $c_j = F$. Otherwise we set $c_j = T$ (note that we never get to this point if one of the three literals from $\phi$ is true).

We can see that under this assignment:

- If $\ell_{j,1} = T$, then the four clauses corresponding to $C_j$ evaluate to

$$(F \vee T \vee F) \wedge (T \vee \ell_{j,2} \vee F) \wedge (T \vee \ell_{j,3} \vee F) \wedge (F \vee T \vee T).$$

- If $\ell_{j,1} = F$ and $\ell_{j,2} = T$, then the four clauses corresponding to $C_j$ evaluate to

$$(F \vee F \vee T) \wedge (F \vee T \vee F) \wedge (T \vee \ell_{j,3} \vee F) \wedge (F \vee T \vee T).$$

- If $\ell_{j,1} = \ell_{j,2} = F$ and $\ell_{j,3} = T$, then the four clauses corresponding to $C_j$ evaluate to

$$(F \vee F \vee T) \wedge (F \vee F \vee T) \wedge (F \vee T \vee F) \wedge (F \vee T \vee T).$$

So all four claues have at least one true and one false literal.

If we do this for every clause $C_j$, we will have a truth assignment $\tau'$ such that for every clause, at least one literal is true and one literal is false.

So $\langle\phi'\rangle \in$NAE-3SAT.

- **Suppose that** $\langle\phi'\rangle \in$NAE-3SAT**:**

Then it has a truth assignment $\tau'$ such that for every clause, at least one literal is true and one literal is false.

Now, we can see that if $\tau'$ is a certificate for $\phi'$, then so is $\neg\tau'$ (that is, the truth assignment obtained by negating every variable assignment in $\tau'$). So we can assume wlog. that in $\tau'$, $x_T = T$.

Let $\tau$ be the truth assignment for $\phi$ that we get by taking the variable assignments from $\tau'$ for the variables $x_i$ in $\phi$.

We argue that $\tau$ is a satisfying truth assignment for $\phi$: that is, for every $C_j$, at least one literal evaluates to true.

Suppose otherwise. Then, there is some $C_j = (\ell_{j,1} \vee \ell_{j,2} \vee \ell_{j,3})$ such that $\ell_{j,1} = \ell_{j,2} = \ell_{j,3} = F$ under $\tau$ (and so under $\tau'$, as well).

Then, we can see that the four $C_j$ clauses in $\phi'$ would be forced to evaluate as follows:

$$\begin{aligned}
&(\neg x_T \vee \ell_{j,1} \vee a_j) \wedge (\neg a_j \vee \ell_{j,2} \vee b_j) \wedge (\neg b_j \vee \ell_{j,3} \vee c_j) \wedge (c_j \vee x_T \vee x_T) \\
=&(F \vee F \vee a_j) \wedge (\neg a_j \vee F \vee b_j) \wedge (\neg b_j \vee F \vee c_j) \wedge (c_j \vee T \vee T) \\
=&(F \vee F \vee T) \wedge (F \vee F \vee b_j) \wedge (\neg b_j \vee F \vee c_j) \wedge (c_j \vee T \vee T) \\
=&(F \vee F \vee T) \wedge (F \vee F \vee T) \wedge (F \vee F \vee c_j) \wedge (c_j \vee T \vee T)
\end{aligned}$$

After which there would be no way to assign a value to $c_j$ without getting either an $(F \vee F \vee F)$ or a $(T \vee T \vee T)$ clause.

This would mean that $\tau'$ was not a certificate for $\phi'$, a contradiction.

So $\tau$ is a satisfying truth assignment for $\phi$, and so $\langle\phi\rangle \in$3SAT.

Finally, we can see that this reduction replaces each of the $n$ clauses of $\phi$ with four new clauses, so that the result is a 3CNF of size $4n$. Each of these clauses can be found in polytime, and so the entire construction is polytime, as required.

Therefore, NAE-3SAT is NP-complete.

■

---

## 3.3 CLIQUE

Definition of CLIQUE:

**Instance:** A graph $G = (V, E)$ and a number $k \in \mathbb{N}$.

**Question:** Does $G$ contain a clique of size $k$ (i.e., a set $C$ of $k$ vertices in $V$ such that, for every $u, v \in C$, where $u \neq v$, $\{u, v\} \in E$)?

**Reduced from:** 3SAT.

Firstly, we can see that this is in NP:

1) It is a yes/no problem.

2) Its certificate is the size-$k$ clique $C$ (we can treat this as a list of the vertices of $V$).

3) The size of this certificate is $\mathcal{O}(n)$, where $n$ is the number of variables in $G$. This certificate is polynomial in the size of $G$.

4) A verifier for this certificate would be:

Let $\mathcal{V}$ ="On $\langle G = (V, E), C\rangle$:"

1. Reject if any of the following checks fails:
   $\mathcal{O}(1)$ *time.*
2. Check that $C$ has at least $k$ and at most $n$ elements.
   $\mathcal{O}(n)$ *time.*
3. For $i = 1$ to $k - 1$:
   $\mathcal{O}(n)$ *iterations.*
4. For $j = i + 1$ to $k$:
   $\mathcal{O}(n)$ *iterations per value of* $i$.
5. Check that $\{C[i], C[j]\} \in E$
   $\mathcal{O}(n)$ *or* $\mathcal{O}(1)$ *time per iteration, depending on your implementation.*
6. *Accept.*
   $\mathcal{O}(1)$ *time.*

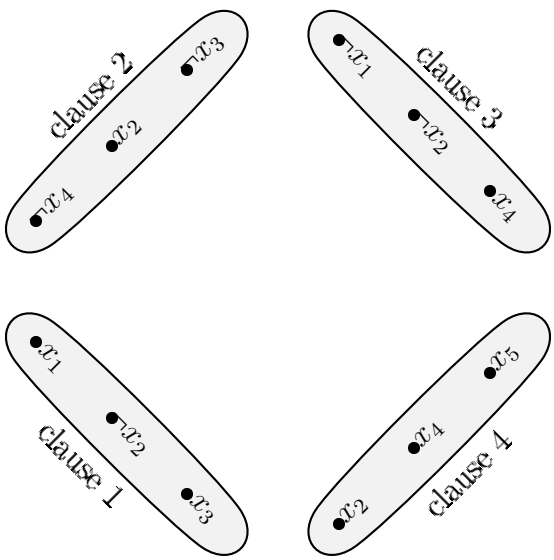
5) We can see that this verifier runs in $\mathcal{O}(n^3)$ time, and so is polytime in the size of $G$.

So CLIQUE is in NP.

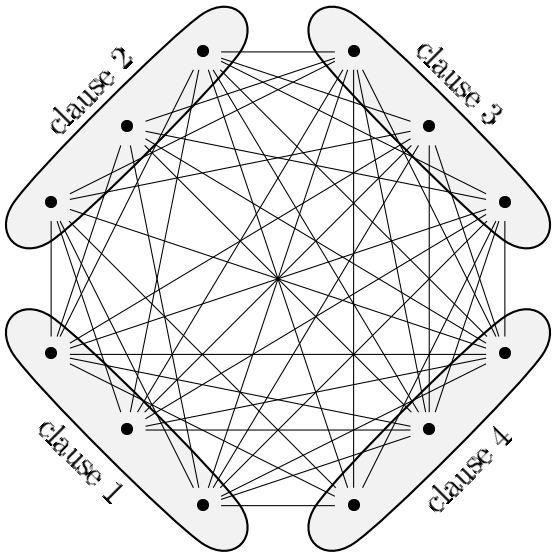
Suppose we are given a 3SAT instance $\phi$ with $\ell$ variables and $k$ clauses.

*e.g.,* $(x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_4 \vee x_2 \vee \neg x_3) \wedge (\neg x_1 \vee \neg x_2 \vee x_4) \wedge (x_2 \vee x_4 \vee x_5)$

**Idea:** Build a $G$ so that each clause maps to a subset of vertices. Each vertex corresponds to one of the three literals in the clause:

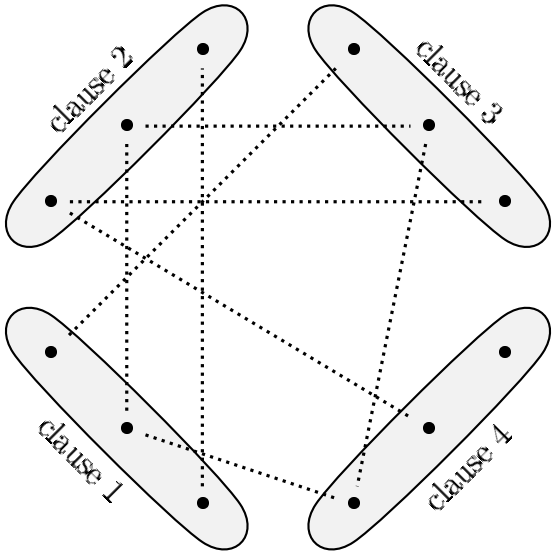



Now, add edges between <u>different</u> subsets whenever those edges could correspond to some consistent truth formula:




*Sorry about how messy it is — that's part of how the reduction works. But look for the edges that are missing.*

**The missing edges:**




This is the $G$ we return (the first one, not the one with the missing edges). To finish, we simply set $k$ to be the number of clauses in $\phi$.

We also note that $G$ has a $k$-clique iff $\phi \in$ 3SAT:

($\Leftarrow$): Suppose $\phi \in$ 3SAT:

- It has a satisfying assignment.
- Let $S$ be a set of vertices in $G$ that takes one vertex from each clause group that matches the truth assignment.
- These variables are connected in $G$.

- $\Rightarrow S$ is a $k$-clique in $G$.

  ($\Rightarrow$): Suppose $G$ has a $k$-clique $C$:

- $C$ must have exactly one vertex in each clause group.
- Every variable node in $C$ must be chosen consistently (i.e., you can't choose $x_1$ in one clause group and $\overline{x_1}$ in another, since there would be no edge between them).
- We can read the truth values of these variables to get a truth assignment. (if a variable is not set by any node in $C$, we can set it however we want).
- Every vertex in $C$ will correspond to a literal that makes its clause true, and so every clause in $\phi$ will be set to true.
- So this is a satisfying assignment for $\phi$.

We note that this construction can be made in polynomial time:

> Given a $\phi$ with $a$ clauses and $b$ variables, the number of vertices in $G$ will be $3a$, and the maximum number of edges will be $9a/2$. We can determine whether each of these edges is in $E$ with at worst $\mathcal{O}(n)$ time, and so the entire reduction is polytime.

So 3SAT $\leqslant_p$ CLIQUE, and so CLIQUE is NP-complete.

■

---

## 3.4 IS

Definition of IS *(Independent Set)*:

> **Instance:** A graph $G = (V, E)$ and a number $k \in \mathbb{N}$.
>
> **Question:** Does $G$ contain an independent set of size $k$ (i.e., a set $I$ of $k$ vertices in $V$ such that, for every $u, v \in I$, where $u \neq v$, $\{u, v\} \notin E$)?

**Reduced from:** CLIQUE.

> Firstly, we can see that this is in NP:
>
> 1) It is a yes/no problem.
> 2) Its certificate is the size-$k$ independent set $I$ (we can treat this as a list of the vertices of $V$).
> 3) The size of this certificate is $\mathcal{O}(n)$, where $n$ is the number of variables in $G$. This certificate is polynomial in the size of $G$.
> 4) A verifier for this certificate would be:

Let $\mathcal{V}$ =“On $\langle G = (V, E), I\rangle$:”

1. Reject if any of the following checks fails:
   $\mathcal{O}(1)$ *time.*
2. Check that $I$ has at least $k$ and at most $n$ elements.
   $\mathcal{O}(n)$ *time.*
3. For $i = 1$ to $k - 1$:
   $\mathcal{O}(n)$ *iterations.*
4. For $j = i + 1$ to $k$:
   $\mathcal{O}(n)$ *iterations per value of* $i$.
5. Check that $\{I[i], I[j]\} \notin E$
   $\mathcal{O}(n)$ *or* $\mathcal{O}(1)$ *time per iteration, depending on your implementation.*
6. *Accept.*
   $\mathcal{O}(1)$ *time.*

5) We can see that this verifier runs in $\mathcal{O}(n^3)$ time, and so is polytime in the size of $G$.

So IS is in NP.

**Reduction from CLIQUE**:

Suppose we are given a CLIQUE instance $\langle G = (V, E), k\rangle$ where $G$ has $n$ vertices and $m$ edges.

We define the **complement** $\overline{G}$ of $G$ to be the graph $(V, \overline{E})$: That is, $\overline{G}$ has the same vertices as $G$, and an edge $\{u, v\}$ is in $\overline{E}$ (is an edge of $\overline{G}$) iff $\{u, v\} \notin E$ (is not an edge of $G$).
We return the pair $\langle \overline{G}, k\rangle$.

We also note that $\overline{G}$ has a size-$k$ independent set iff $G$ has a $k$-clique:

($\Leftarrow$): Suppose $G$ has a $k$-clique $C$:

- Then for any $u \neq v \in C$, $\{u, v\} \in E$
- $\Rightarrow \{u, v\} \notin \overline{E}$
- $\Rightarrow C$ is a size-$k$ independent set in $\overline{G}$.

($\Rightarrow$): Suppose $\overline{G}$ has a size-$k$ independent set $I$: Then for any $u \neq v \in I$, $\{u, v\} \notin \overline{E}$

- $\Rightarrow u \neq v \in E$
- $\Rightarrow I$ is a $k$-clique in $G$.

So CLIQUE $\leqslant_p$ IS, and so IS is NP-complete.

■

---

## 3.5 VC

Definition of VC *(Vertex Cover)*:

**Instance:** A graph $G = (V, E)$ and a number $k \in \mathbb{N}$.

**Question:** Does $G$ contain a vertex cover of size $k$ or less (i.e., a set $V'$ of $k$ vertices in $V$ such that, for every edge $\{u, v\} \in E$, at least one of $u$ or $v$ is in $V'$)?

**Reduced from:** IS.

Firstly, we can see that this is in NP:

1) It is a yes/no problem.
2) Its certificate is the size-$k$ vertex cover $V'$ (we can treat this as a list of the vertices of $V$).
3) The size of this certificate is $\mathcal{O}(n)$, where $n$ is the number of vertices in $G$. This certificate is polynomial in the size of $G$.
4) A verifier for this certificate would be:

   Let $\mathcal{V}$ ="On $\langle G = (V, E), V' \rangle$:"
   1. Reject if any of the following checks fails:
      $\mathcal{O}(1)$ *time.*
   2. Check that $V'$ has at most $\min(k, n)$ elements.
      $\mathcal{O}(n)$ *time.*
   3. For all edges $\{u, v\} \in E$:
      $\mathcal{O}(n^2)$ *iterations.*
   4. Check that at least one of $u$ or $v$ is in $V'$.
      $\mathcal{O}(n)$ *time per iteration.*
   5. *Accept.*
      $\mathcal{O}(1)$ *time.*
5) We can see that this verifier runs in $\mathcal{O}(n^3)$ time, and so is polytime in the size of $G$.

So VC is in NP.

**Reduction from IS**:

Suppose we are given an IS instance $\langle G = (V, E), k \rangle$ where $G$ has $n$ vertices and $m$ edges.

We return the pair $\langle G, n - k \rangle$.

Note that $G$ has a size-$k$ independent set iff it also has a size-$(n - k)$ vertex cover:

($\Leftarrow$): Suppose $G$ has a size-$k$ independent set $I$:

- Then for any $u \neq v \in I$, $\{u, v\} \notin E$
- $\Rightarrow$ For any $\{u, v\} \in E$, at least one of $u$ or $v$ is in $V \setminus I$.
- $\Rightarrow V \setminus I$ is a size-$(n - k)$ vertex cover in $G$.

($\Rightarrow$): Suppose $\overline{G}$ has a size-$(n-k)$ vertex cover $V'$: Then for any $\{u, v\} \in E$, at least one of $u$ or $v$ is in $V'$.

- Then for any $u \neq v \in V \setminus V'$, $\{u, v\} \notin E$
- $\Rightarrow I$ is a size-$(n-(n-k)) = k$ independent set in $G$.

So IS $\leqslant_p$ VC, and so VC is NP-complete.

■

---

## 3.6 3COL

Definition of 3COL *(3-Colourability)*:

**Instance:** A graph $G = (V, E)$.

**Question:** Does $G$ admit a three (vertex) colouring (i.e., a way of 3-colouring the vertices of $G$ so that no edge has two endpoints of the same colour)?

**Reduced from:** 3SAT.

Firstly, we can see that this is in NP:

1) It is a yes/no problem.
2) Its certificate is the 3-colouring $C$ (we can treat this as a dictionary or mapping of the vertices of $V$ to $\{$*red, green, blue*$\}$).
3) The size of this certificate is $\mathcal{O}(n)$, where $n$ is the number of vertices in $G$. This certificate is polynomial in the size of $G$.
4) A verifier for this certificate would be:

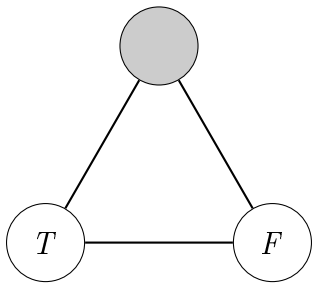
   Let $\mathcal{V}$ =“On $\langle G = (V, E), C\rangle$:”
   1. Reject if any of the following checks fails:
      $\mathcal{O}(1)$ *time.*
   2. Check that $C$ maps every vertex to one of three colours (we'll refer to them as $\{$*red, green,* and *blue*$\}$).
      $\mathcal{O}(n)$ *time.*
   3. For all edges $\{u, v\} \in E$:
      $\mathcal{O}(n^2)$ *iterations.*
   4. Check that $C[u] \neq C[v]$.
      $\mathcal{O}(n)$ *time per iteration.*
   5. *Accept.*
      $\mathcal{O}(1)$ *time.*
5) We can see that this verifier runs in $\mathcal{O}(n^3)$ time, and so is polytime in the size of $G$.

So 3COL is in NP.

Suppose we are given a 3SAT instance $\phi$ with $\ell$ variables and $k$ clauses.

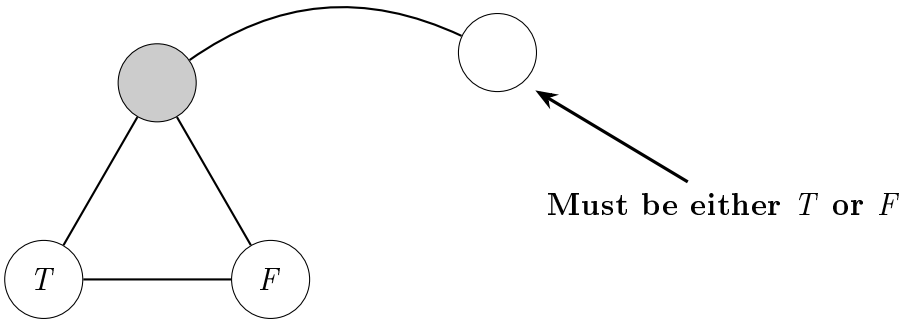
The trick here is to build graph widgets that actually encode the individual *or*s, *not*s, and *and*s from the Boolean formula.

To start, let's build a palette widget for our graph. This is actually a very simple widget: it's just a triangle.

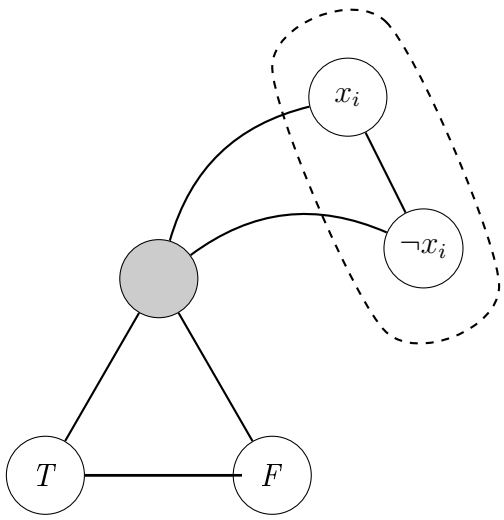



You can see that any 3-colouring of a graph with this widget must give a different one of the three colours to each of these three nodes.
Now, if we're able to 3-colour a graph in one way, we know at least five other ways to colour it as well: we can always permute the colours. So the colours aren't fixed.
But for the moment we'll treat them as if the colour are the ones we see here - we'll label them *true*, *false*, and *red*.

You can see that, by attaching the *red* palette node to any other node, we can force that other node to take one of the two Boolean values.

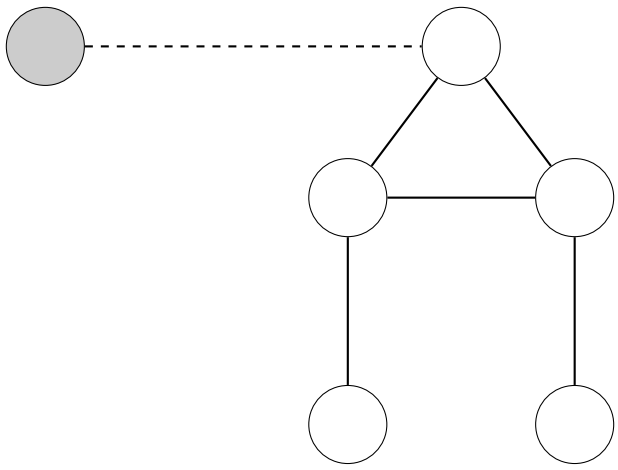



Using this trick, the first thing we want to encode are the variable assignments for $\phi$. We can do this by creating a node for every variable $x_i$, and for its negation $\neg x_i$. If we connect both to *red*, they must take values of *true* or *false*, and if we connect them to each other they must take opposite values.

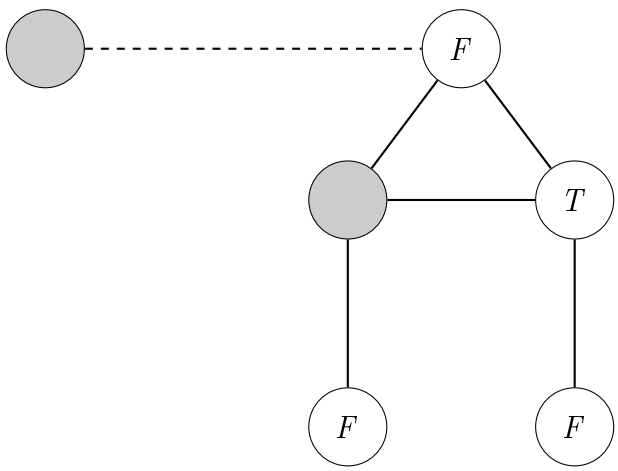



We can repeat this process for every variable.

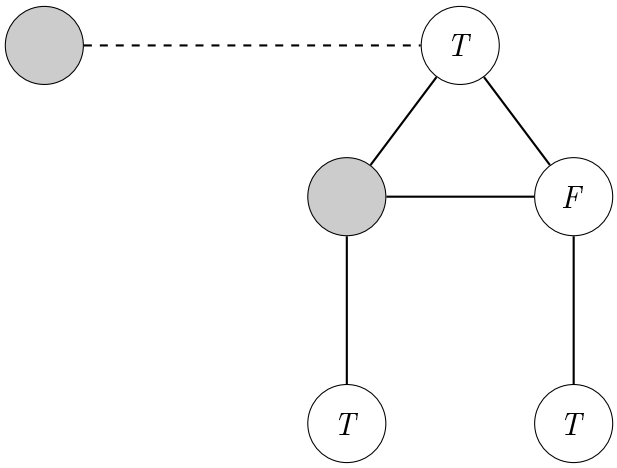
Now, we can build an *or* widget for our graph:



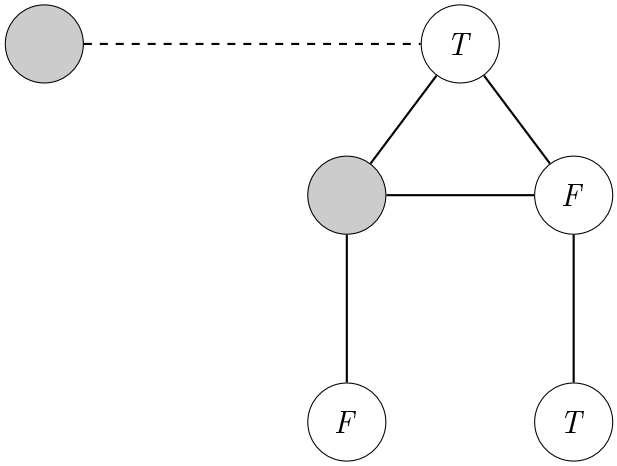
The two bottom nodes will be Boolean valued - they'll have a colour of either *true* or *false*. If we do this, you can see that the top node can take the value *true* iff one of the two bottom nodes also has a value true: if both are *false*, the only way to fill colour the widget is



*(Or the mirror image.)*
On the other hand, the colourings

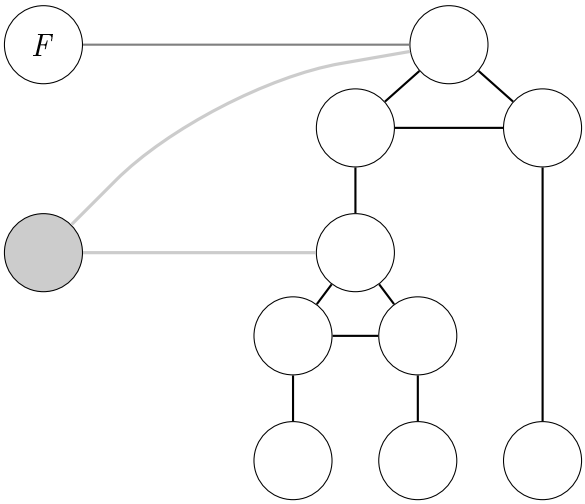



and



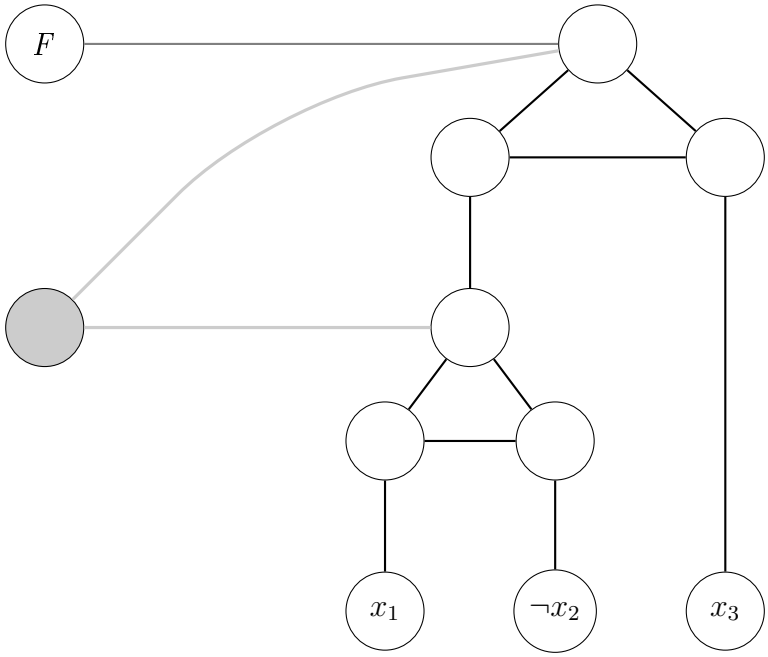

*(Again, and their mirror images.)*
Are all possible. So it is possible to assign a *true* to the top node if and only if at least one bottom node is *true*, as desired.

Now, for a 3CNF formula, each clause is made up of two *or* operations on some literals. We can simulate this by stacking two of these widgets on top of each other:



From our discussion of the *or*-widget, we know that the top node can be coloured *true* iff at least one of the bottom nodes is also *true*, but the fact that we have connected this node to *red* and *false* means that this is the only colouring that we can accept.

If we identify the bottom nodes with the literals in a clause, then, we will have encoded the whole clause into the 3SAT problem:




If we repeat this process for every clause, we will have our entire graph, and, by our discussion, this graph will be 3-colourable iff the input $\phi$ was satisfiable.

$\Rightarrow$) If $\phi$ has a satisfying truth assignment, then we can use the values from that assignment to fill in the inputs to the corresponding *or*-widgets. By our discussion above, these widgets can then be 3-coloured.

$\Leftarrow$) If our output graph $G$ is 3-colourable, then we can argue wlog. that its palette widget has the *red*/T/F colouring we used above (if not, we can simply rename the colours).

Since the inputs to the *or*-widget are connected to the *red* node in the palette, they must be either T of F. We can read these values to get a truth assignment $\tau$ for $\phi$. Since each *or*-widget is satisfied iff their inputs are not all F, this truth assignment must be a satisfying truth assignment for $\phi$.

Finally, we can see that for an $\ell$-variable, $k$-node $\phi$, this graph requires 3 nodes for the palette, two nodes for each of the $\ell$ variables, and six nodes per clause.

Likewise, we need three edges for the palette, three for each variable, and eleven for each clause.

So all told, the number of vertices and edges in this graph are both $\mathcal{O}(\ell + k)$, and we can construct the edges and vertices using simple lookups to the input $\phi$. So the whole construction is polytime, as required.
So 3SAT $\leqslant_p$ 3COL, and so 3COL is NP-complete.

■

## 3.7 HAM-PATH

Definition of HAM-PATH *(Hamiltonian Path)*:

> **Instance:** A directed graph $G = (V, E)$ with specified vertices $s$ and $t$.
>
> **Question:** Does there exist a Hamiltonian path in $G$ from $s$ to $t$ (i.e., a path from $s$ to $t$ that passes through every $v \in V$ exactly once)?

**Reduced from:** 3SAT.

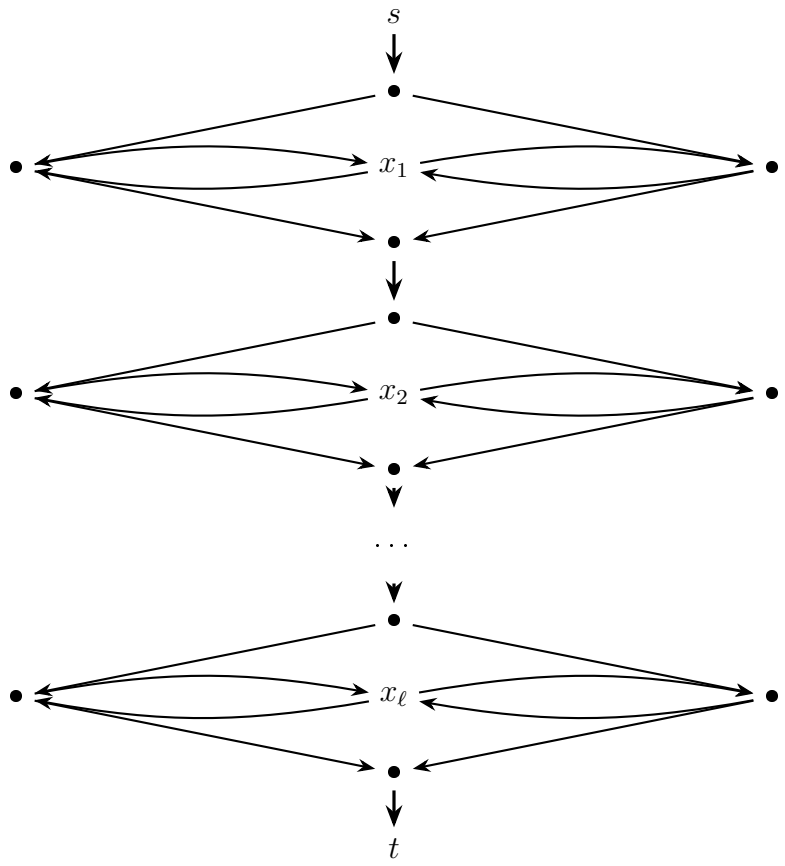
Firstly, we can see that this is in NP:

1) It is a yes/no problem.
2) Its certificate is a Hamiltonian path $P$ from $s$ to $t$ (we can treat this as a list of the vertices of $V$).
3) The size of this certificate is $\mathcal{O}(n)$, where $n$ is the number of vertices in $G$. This certificate is polynomial in the size of $G$.
4) A verifier for this certificate would be:

   Let $\mathcal{V}$ =“On $\langle G = (V, E), s, t, P\rangle$:”
   1. Reject if any of the following checks fails:
      $\mathcal{O}(1)$ *time.*
   2. Check that $P$ has exactly $n = |V|$ distinct elements.
      $\mathcal{O}(n^2)$ *time.*
   3. For $i = 1$ to $n - 1$:
      $\mathcal{O}(n)$ *iterations.*
   4. Check that $(P[i], P[i + 1]) \in E$.
      $\mathcal{O}(n)$ *or* $\mathcal{O}(1)$ *time per iteration, depending on your implementation.*
   5. Check that $P[1] = s$ and $P[n] = t$.
      $\mathcal{O}(1)$ *time.*
   6. *Accept.*
      $\mathcal{O}(1)$ *time.*
5) We can see that this verifier runs in $\mathcal{O}(n^2)$ time, and so is polytime in the size of $G$.

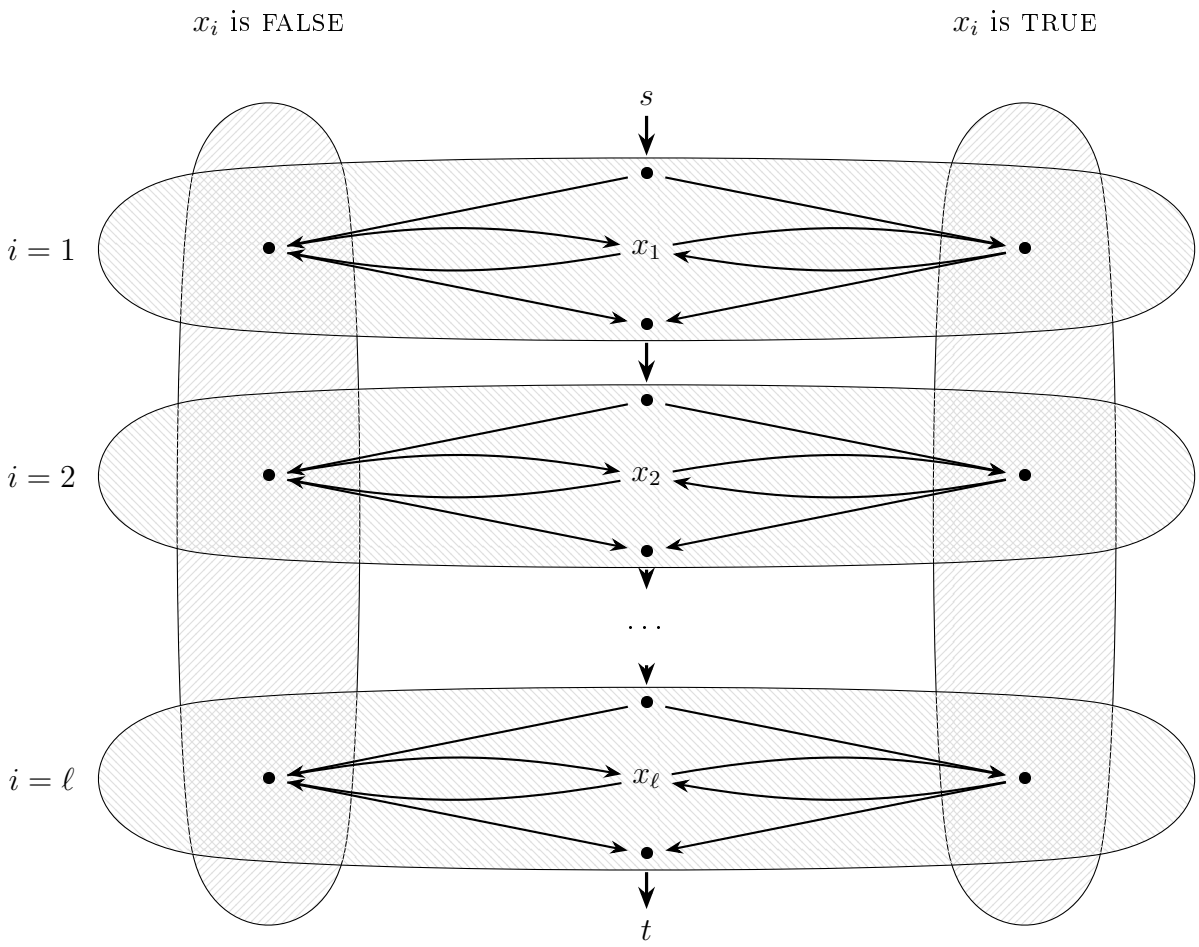
So HAM-PATH is in NP.

Suppose we are given a 3SAT instance $\phi$ with $\ell$ variables and $k$ clauses. Note that we can assume using the lemma in section 2 that the clauses in $\phi$ each have three distinct variables.
We'll start by encoding a choice of truth assignment in $\phi$ into a HAM-PATH instance.
We'll do this with the following graph:

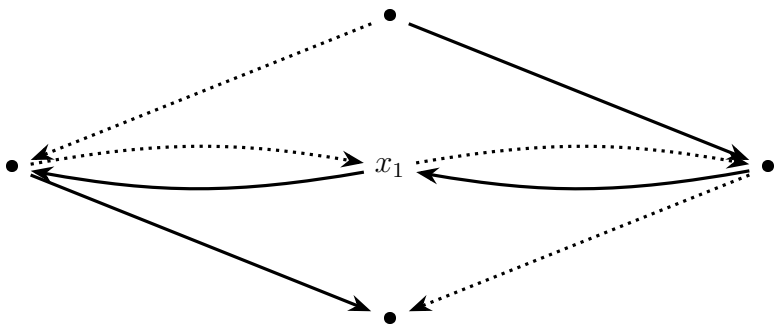



*OK, so what does this do?*

Well, each "level" of the graph represents a variable, and the choice of which direction you move on that variable represents whether it is set to true or false:




That is, if we expand out the top level, we can see that there are exactly two possible ways that a Hamiltonian path can pass through it:



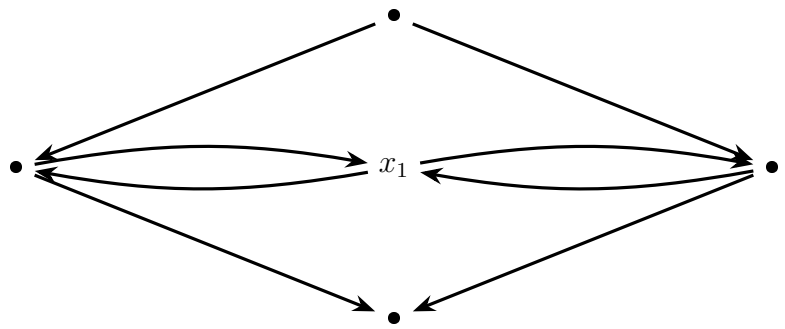

We'll treat the dotted line as an assignment of FALSE to $x_1$, and the solid line as an assignment of TRUE.

The same idea applies to every variable: every Hamiltonian path through our full graph corresponds to a truth assignment to our initial 3SAT fomula $\phi$.

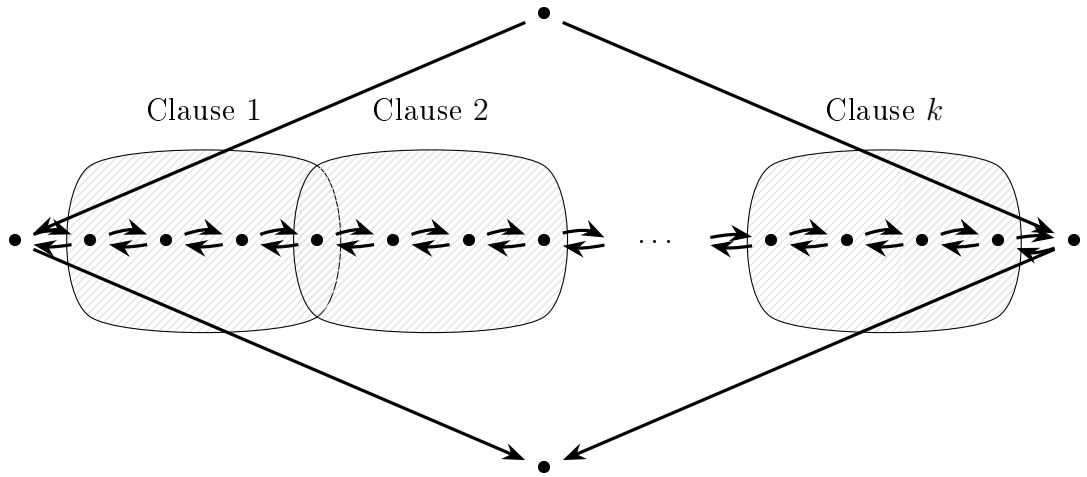
Now, we need to fill in the clauses. We'll do that by expanding the $x_1, x_2, \ldots, x_\ell$ vertices in the center of the graph.

The first thing we'll do is break the $x_i$ nodes into a series of widgets, one per clause.

For example, when we have $k$ clauses, the first level of our graph will change from



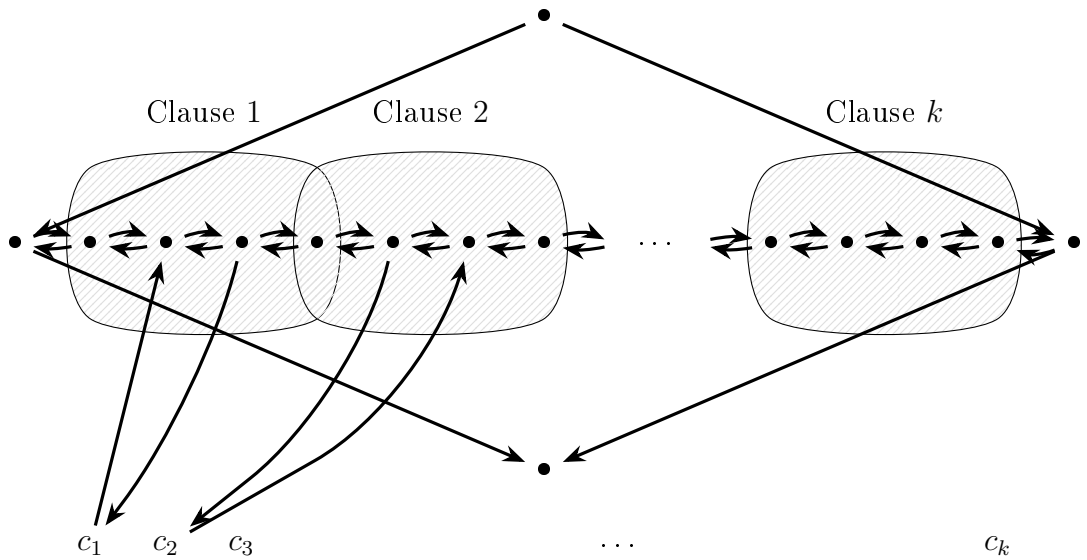

to




*The shared node between clause 1 and clause 2 is the exit from one widget and the entrance to the next.*

Remember that the direction of travel across the entire diamond is based on whether I have set $x_1$ to TRUE or FALSE: a left-to-right movement across the clauses indicates $x_1$ is FALSE, a right-to-left movement indicates a TRUE.

With that in mind, we'll repeat this process with every variable, and then we'll add a new set of nodes to the graph: one per clause:



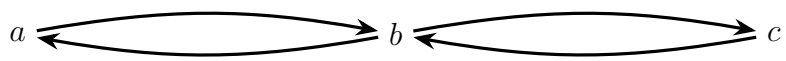
There is one of each vertex $c_1$, $c_2$, ..., $c_k$ in the graph - they are not repeated between variables. You'll see that the clause 1 and clause 2 widgets are attached to their respective clauses. The leftward movement of the clause 1 arrows again indicates that $x_1$ is TRUE - in this case, we put these arrows in if and only if $x_1$ is in clause 1. Similarly, the rightward movement of the clause 2 arrows indicates that $x_2$ is FALSE - in this case, we put these arrows in if and only if $\neg x_1$ is in clause 2.

*If a clause has both $x_i$ and $\neg x_i$ in it, it will always be satisfied, so we cn safely ignore it during the reduction.*

Once we've done this for every variable and every clause, we're done the reduction. As a check, you'll have thee in arrows and three out arrows from each clause vertex.

$\Rightarrow$) Now, you can see that if there is any satisfying assignment for our input $\phi$, taking the corresponding direction on each of the diamond variable widgets will allow us to visit each clause - so we will be able to form a Hamiltonian path from $s$ to $t$.

$\Leftarrow$ On the other hand, these are the only possible Hamiltonian paths: consider the node $b$ in the following graph:




*In this subgraph, a and c may be connected to other vertices, but b may not.*

If we visit the node $a$ but do not continue on to $b$, there will be no way to visit $b$ - its only other neighbour is $c$. If we were to use $c$ later in order to get to $b$, we would be stuck at $b$: both of its neighbours would have already been visited.

But you can see in the reduction that every clause vertex is hedged on both sides by a subgraph of this form. If we were to try to enter a clause node from one variable and leave using another, or if we were to try to go through the clause vertex backwards, we'd end up with an unusable $b$, just like in the subgraph above. So in any Hamiltonian cycle, the path passing through any clause node must leave in the same variable and in the same direction as it started in. This means that every Hamiltonian path corresponds to a satisfying truth assignment for $\phi$.

Finally, we see that for an $\ell$-variable, $k$-clause $\phi$, this graph has $4k+3$ nodes per variable widget, and $\ell$ of these widgets. There are an additional $k$ clause nodes, and two more nodes for $s$ and $t$. So the total number of nodes is $\ell(4k+3)+k+2$. Each variable widget has $8k+4$ edges, again multiplied by $\ell$, and each of the $k$ clause nodes is an endpoint 6 additional edges. Finally, we need an additional $\ell+1$ edges to connect the $s$, the $t$, and the variable widgets. So the total number of edges is $\ell(8k+4)+6k+\ell+1$. Altogether, the number of vertices and edges in the graph are both $\mathcal{O}(\ell k)$, and each vertex and edge can be constructed using a lookup in the input $\phi$. So the total reduction is polytime, as required.
So 3SAT $\leqslant_p$ HAM-PATH, and so HAM-PATH is NP-complete.

■

## 3.8 HAM-CYCLE

Definition of HAM-CYCLE *(Hamiltonian Cycle)*:

**Instance:** A directed graph $G = (V, E)$.

**Question:** Does there exist a Hamiltonian cycle in $G$ (i.e., a cycle that passes through every $v \in V$ exactly once)?

**Reduced from:** HAM-PATH.

Firstly, we can see that this is in NP:

1) It is a yes/no problem.
2) Its certificate is a Hamiltonian cycle $C$ (we can treat this as a list of the vertices of $V$).
3) The size of this certificate is $\mathcal{O}(n)$, where $n$ is the number of vertices in $G$. This certificate is polynomial in the size of $G$.
4) A verifier for this certificate would be:

   Let $\mathcal{V}$ ="On $\langle G = (V, E), C\rangle$:"

   1. Reject if any of the following checks fails:
      $\mathcal{O}(1)$ *time.*
   2. Check that $C$ has exactly $n = |V|$ distinct elements.
      $\mathcal{O}(n^2)$ *time.*
   3. For $i = 1$ to $n - 1$:
      $\mathcal{O}(n)$ *iterations.*
   4. Check that $(C[i], C[i + 1]) \in E$.
      $\mathcal{O}(n)$ *or* $\mathcal{O}(1)$ *time per iteration, depending on your implementation.*
   5. Check that $(C[n], C[1]) \in E$.
      $\mathcal{O}(n)$ *or* $\mathcal{O}(1)$ *time, depending on your implementation.*
   6. *Accept.*
      $\mathcal{O}(1)$ *time.*
5) We can see that this verifier runs in $\mathcal{O}(n^2)$ time, and so is polytime in the size of $G$.

So HAM-CYCLE is in NP.

**Reduction from HAM-PATH**:

Suppose we are given an instance $\langle G = (V, E), s, t\rangle$ of HAM-PATH.

Let $V' = V \cup \{v'\}$, where $v'$ is a new vertex not in $V$. We'll set $E' = E \cup \{(t, v'), (v', s)\}$. Let $G' = (V', E')$.

We argue that $\langle G'\rangle \in$ HAM-CYCLE iff $\langle G, s, t\rangle \in$HAM-PATH:

Suppose $\langle G, s, t\rangle \in$HAM-PATH

Then there is a Hamiltonian path $P = (v_1 = s, v_2, v_3, \ldots, v_{n-1}, v_n = t)$ in $G$.

But then the cycle $P = (v', v_1 = s, v_2, v_3, \ldots, v_{n-1}, v_n = t)$ encodes a Hamiltonian cycle in $G'$.

$\Rightarrow \langle G'\rangle \in$ HAM-CYCLE.

Suppose $\langle G' \rangle \in$ HAM-CYCLE.

Then there is a Hamiltonian cycle $C = (v', v_1, \ldots, v_n)$ in $G'$.

Since the only edge out of $v'$ is $(v', s)$, it must hold that $v_1 = s$.

Similarly, the only edge into $v'$ is $(t, v')$, and so $v_n = t$.

But then $P = (s = v_1, \ldots, t = v_n)$ is a Hamiltonian path from $s$ to $t$ in $G$.

$\Rightarrow \langle G, s, t \rangle \in$HAM-PATH.

Finally, we see that our reduction simply copies out $G$ with the addition of one vertex and two edges. We can determine where to place this vertex and these edges in polynomial time, and so the entire reduction is polytime.

So HAM-CYCLE is NP-complete, as required.

■

---

## 3.9 INDUCED-PATH

Definition of INDUCED-PATH:

**Instance:** A graph $G = (V, E)$ and a number $k \in \mathbb{N}$.

**Question:** Is there a subset $V' \subseteq V$ such that $|V'| \geqslant k$ and such that the subgraph induced by $V'$ is a simple path on $|V'|$ vertices?

**Reduced from:**3-SAT.

Firstly, we can see that this is in NP:

1) It is a yes/no problem.
2) Its certificate is the set $V' \subseteq V$.
3) The size of this certificate is $\mathcal{O}(V)$, and so this certificate is polynomial in the size of the input.
4) A verifier for this certificate would be:

Let $\mathcal{V}$ =“On $\langle G = (V, E), k, V' \rangle$:”

1. Reject if any of the following checks fails:
   $\mathcal{O}(1)$ *time.*
2. Check that $V' \subseteq V$.
   $\mathcal{O}(|V|^2)$ *time.*
3. Check that $|V'| \geqslant k$.
   $\mathcal{O}(|V|)$ *time.*
4. Let $G'$ be the subgraph of $G$ induced by $V'$.
   $\mathcal{O}(|V|^2)$ *or* $\mathcal{O}(|V|^3)$ *time to construct, depending on your implementation.*
5. Check that $G'$ is connected and has no vertex of degree more than two (i.e., that it's a path).
   $\mathcal{O}(|V|^2)$ *time.*
6. *Accept.*
   $\mathcal{O}(1)$ *time.*

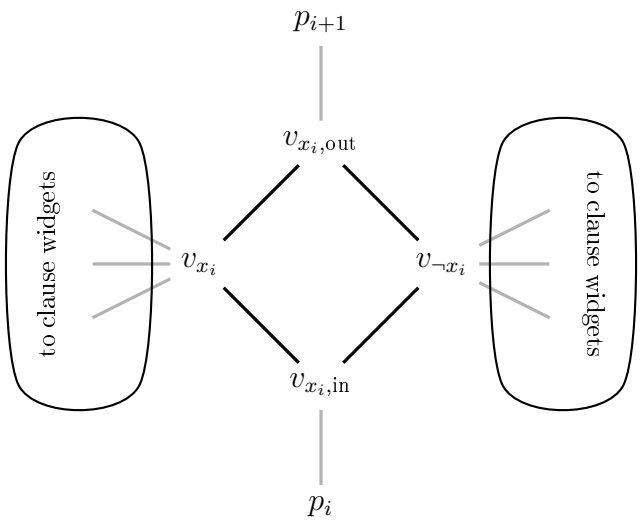
5) We can see that this verifier runs in $\mathcal{O}(|V|^3)$ time, and so is polytime in the size of $G$.

So INDUCED-PATH is in NP.

**Reduction from 3SAT**:
Let an input 3CNF $\phi$ be given, where $\phi$ has $m$ variables and $n$ clauses. Note that we can assume using the lemma in section 2 that the clauses in $\phi$ each have three distinct variables.

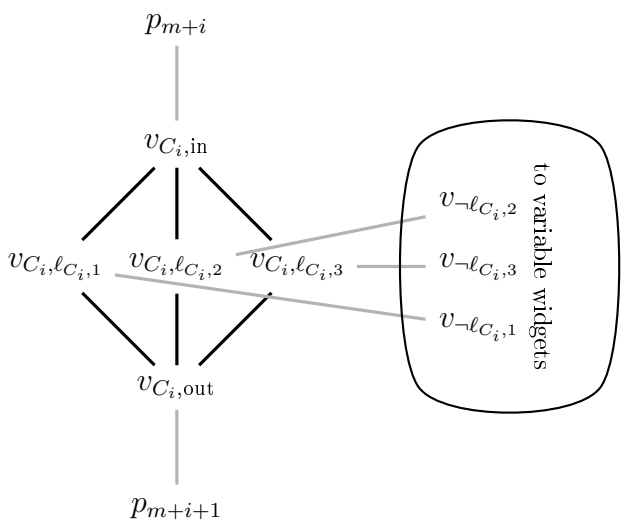
We build $G$ as follows: we start by building $m + n + 1$ copies of a single vertex. We'll label these vertices as $p_1$ to $p_{m+n+1}$.

We'll attach $p_i$, $1 \leqslant i \leqslant m$, with the following widget for the $m$ vertices in $\phi$:




Note that we attach $p_{i+1}$ to the output (the top) of each variable widget.

Once we're done with the variable widgets, we'll start on the clause widgets. As before, we'll attach $p_{m+i}$ to the input of the clause widget for clause $i$, and we'll attach the output of the clause widget to $p_{m+i+1}$. The clause widget for clause $C_i = (\ell_1 \vee \ell_2 \vee \ell_3)$ will be as follows:



So we connect the vertex for each literal $\ell_{C_i,a}, a \in \{1,2,3\}$ in $C_i$ with the input and output of the clause widget, but also to the *negation* of $\ell_{C_i,a}$ in the varibale widgets: so if one of the literals was $x_1$, we would connect to $v_{\neg x_1}$ in the first variable widget. Similarly, if one of the literals is $\neg x_2$, we would connect to the $v_{x_2}$ in the second variable widget.

Once we've connected all of our variable and clause widgets we have our graph $G$. We'll set $k = 3m + 3n + (m + n + 1) = 4m + 4n + 1$.

We argue that $\langle G, k\rangle \in$INDUCED-PATH iff $\langle \phi\rangle \in$3SAT:

Suppose $\langle \phi\rangle \in$3SAT.

Then $\phi$ has a satisfying truth assignment $\tau$.

So we can pick, for any clause $C_i$, a literal $\ell_{C_i,a}, a \in \{1,2,3\}$, which is satisfied by $\tau$.

Consider the following $V'$:

- Every $p_{i'}, 1 \leqslant i' \leqslant m + n + 1$ is contained in $V'$.
- For every variable $x_i$, the nodes $v_{x_i,\text{in}}$ and $v_{x_i,\text{out}}$ are in $V'$.

  If $\tau$ sets $x_i$ to true, the vertex $v_{x_i} \in V'$, while if $x_i = F$ under $\tau$ the vertex $v_{\neg x_i} \in V'$.
- For every clause $C_j$, we include the vertices $v_{C_j,\text{in}}$ and $v_{C_j,\text{out}}$ in $V'$.

  Moreover, we add the vertex $v_{C_i,\ell_{C_i,a}}$ to $V'$ for one literal set to true by $\tau$.

We can see that $|V'| = 3m + 3n + (m + n + 1) = 4m + 4n + 1 = k$, and so it satisfies the size requirement for INDUCED-PATH.

Moreover, we can see that the graph $G'$ induced by $V'$ is connected – there is a path through every vertex and clause widget, as well as through all of the $p_{i'}$ In fact, $G'$ has a path through all of these widgets.

We can also see that $G'$ has no possible branching points or cycles: the $p_{i'}$ connect in $G$ only to their neighbouring widgets, all of which contain simple paths. No two clause widgets are adjacent to each other in $G$, nor are any variable widgets adjacent.

The only possible adjacencies are between the clause and the vertex widgets. And we can see that we can only get an adjacency between nodes in these widgets if we choose a literal in some $C_j$ that is not satisfied by $\tau$ (e.g., if $C_j = (x_1 \vee x_2 \vee x_3)$ and we set $x_1 = F$ and then choose $v_{C_j,\ell_{C_j,1}}$ in our $V'$).

Since we have avoided these adjacencies in our construction of $V'$, we can see that $V'$ induces a simple path on $G$.

$\Rightarrow \langle G, k\rangle \in$INDUCED-PATH.

Suppose $\langle G, k\rangle \in$INDUCED-PATH.

Then there is some $V'$ of size at least $k = 3m + 3n + (m + n + 1)$ that induces a path $P$ on $G$.

Note that we cannot use more than three vertices in any one vertex widget without inducing a cycle, and that we cannot use more than three vertices in any clause widget without inducing either a cycle or a claw in our graph. So these widgets can add at most three vertices each to $P$.

So at most $3m + 3n$ vertices are in these widgets, and the remaining $(m + n + 1)$ vertices are in the vertices $p_i$. Since there are exactly $(m + n + 1)$ vertices of this type, they must all be contained in $P$, and so the clause and variable widgets must contain three vreices each in $P$.

Now, we can see that $p_1$ and $p_{m+n+1}$ have a degree of one even in $G$, and so they must be the beginning and end of $P$. This path is undirected, but we'll treat $p_1$ as its beginning.

Now, each $p_i$ must connect to either one or two other vertices in $V'$, and since $p_1$ and $p_{m+n+1}$ are the beginning and end points in $P$, they must be the two $p_i$ of degree one. All other $p_i$ must connect to two other vertices in $V'$.

But the only way these vertices can connect to anything is through the $v_{x_i,\text{in}}$, $v_{x_i,\text{out}}$, $v_{C_j,\text{in}}$, and $v_{C_j,\text{out}}$ vertices in the variable and clause widgets. Moreover, the only way we can have a degree of one for $p_1$ and $p_{m+n+1}$ and a degree of two for all other $p_i$ is if all of the $v_{x_i,\text{in}}$, $v_{x_i,\text{out}}$, $v_{C_j,\text{in}}$ are included in $V'$.

Since each variable widget has exactly one more vertex in $V'$, and since these vertices are either of the form $v_{x_i}$ or $v_{\neg x_i}$, we can use these vertices to determine a truth assignment $\tau$: $\tau$ sets $x_i = T$ if $v_{x_i} \in V'$, otherwise it sets it to $F$.

We also see that there must be some $v_{C_j,\ell_{C_j,a}}$ variable included in $V'$ for every clause $C_j$. As we have previously noted, if the literal $\ell_{C_j,a}$ were not satisfied by $\tau$, then there would be an edge between $v_{C_j,\ell_{C_j,a}}$ and $v_{\neg\ell_{C_j,a}}$. This edge would induce a cycle on $G$ when paired with the rest of $V'$.

So $\ell_{C_j,a}$ must be satisfied by $\tau$.

Since this is true for every clause $C_j$, $\tau$ must set at least one variable in each clause to true.

$\Rightarrow \langle\phi\rangle \in$3SAT.

Finally, we see that this graph uses $4m + 5n + (m + n + 1) = \mathcal{O}(m + n)$ vertices, and that the number of edges is also $\mathcal{O}(m + n)$. The values for these vertices and edges can be calculated in polynomial time, and so this entire reduction is polytime.

So INDUCED-PATH is NP-complete, as required.

■

## 3.10 PARTITION-INTO-TRIANGLES

Definition of PARTITION-INTO-TRIANGLES

**Instance:** A graph $G = (V, E)$.

**Question:** Can $G$ be partitioned into disjoint triangles (i.e., can the vertices in $V$ be split into pairwise disjoint sets of three such that each set is a trinagle in $G$)?

**Reduced from:** 3SAT.

Firstly, we can see that this is in NP:

1) It is a yes/no problem.
2) Its certificate is the partitioning $P$ (we can treat this as a collection of triples of vertices in $V$).
3) The size of this certificate is $\mathcal{O}(n)$, where $n$ is the number of vertices in $G$. This certificate is polynomial in the size of $G$.
4) A verifier for this certificate would be:

   Let $\mathcal{V}$ =“On $\langle G = (V, E), P\rangle$:”

   1. Reject if any of the following checks fails:
      $\mathcal{O}(1)$ *time.*
   2. Check that $P$ lists each $v \in V$ exactly once in its triples, and that there are no extra vertices in $P$).
      $\mathcal{O}(n^2)$ *time.*
   3. For all triples $\{a, b, c\} \in P$:
      $\mathcal{O}(n)$ *iterations.*
   4. Check that $\{a, b\}$, $\{a, c\}$, and $\{b, c\}$ are all edges in $E$.
      $\mathcal{O}(n)$ *time per iteration.*

   6.*Accept.*
      $\mathcal{O}(1)$ *time.*
5) We can see that this verifier runs in $\mathcal{O}(n^2)$ time, and so is polytime in the size of $G$.

So PARTITION-INTO-TRIANGLES is in NP.

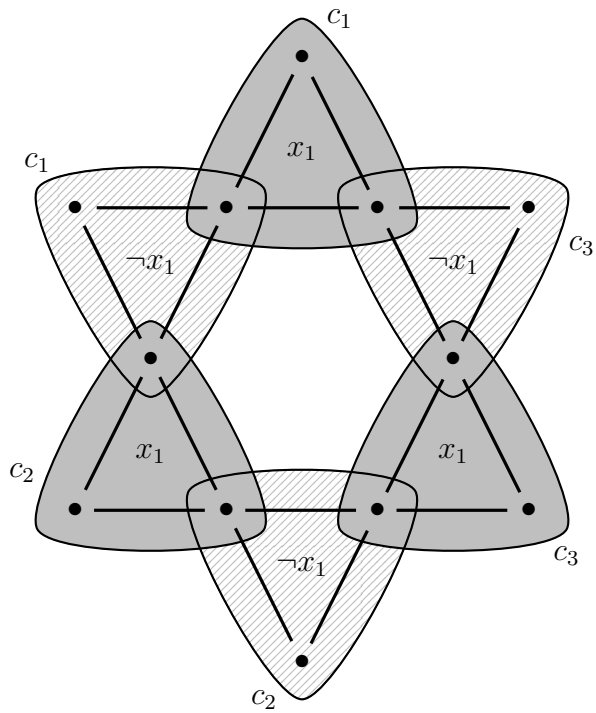
**Reduction from 3SAT**:

Suppose we are given a 3SAT instance $\phi$ with $\ell$ variables and $k$ clauses. Note that we can assume using the lemma in section 2 that the clauses in $\phi$ each have three distinct variables.

Firstly, here's a widget for truth assignments. Suppose (as an example) we have a formula

$$\phi = (x_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \neg x_2 \vee x_3) \wedge (\neg x_1 \vee \neg x_2 \vee \neg x_3)$$

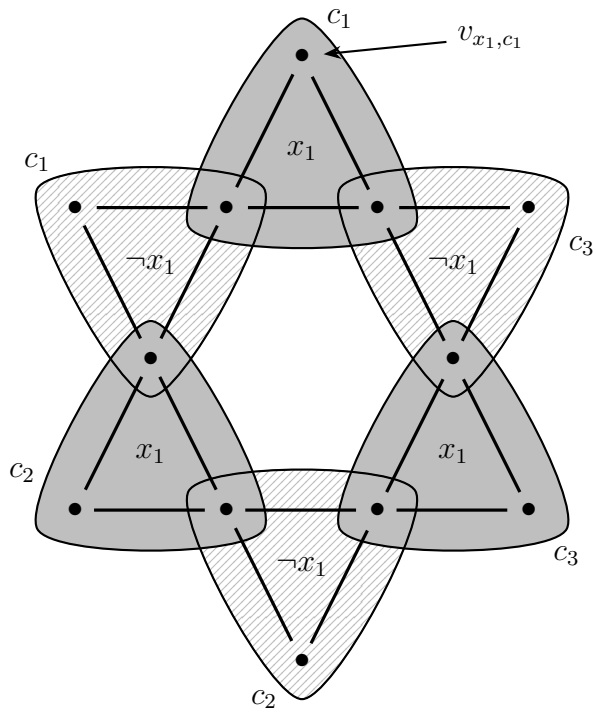
We'll create the following widget for all of the variables (this one is for $x_1$):




So we have two triangles for each clause in $\phi$. The inner nodes don't connect to anything else, and the outer nodes can be connected. The same construction can be used for different $\phi$, with the understanding that the star will have two leaves per clause in $\phi$

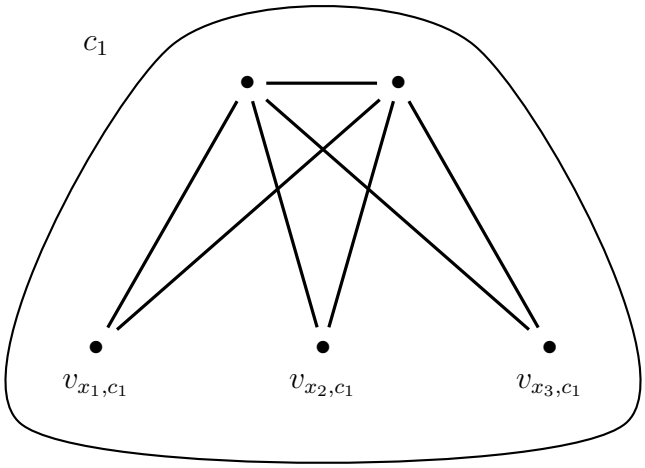
You can see that if we try to break this graph into triangles, there are two ways of doing it — we can choose the ones labeled $x_1$ (the solid grey), or we can choose the ones labeled $\neg x_1$ (the striped ones).

Using this widget, the star widget for $x_1$ will have a solid-coloured leaf for $x_1$ and the clause $c_1$. We will refer to this node as $v_{x_1,c_1}$:



We can similarly find the nodes $v_{\neg x_2,c_1}$ and $v_{x_3,c_1}$ for variables $x_2$ and $x_3$.

With that in mind, our widget for the clause $c_1$ would be




You can see that the only way to build a triangle using the top two nodes is to connect it to one of the three nodes in the variable assignment widgets.

Now, if we have a $\phi$ with $n$ clauses and $k$ variables, our truth assignment widgets will have $2nk$ outer nodes, half of which will be covered by setting a truth assignment to the variables. We can assign $n$ mode of these outer nodes to triangles if we can set every clause to true, leaving $(k-1)n$ of these outer nodes remaining.

We deal with these nodes by building $k(n-1)$ extra copies of the clause widgets, with the change that they connect to every outer node in our variable widget.

Once we've connected all of these widgets together we have our output graph $G$.
**Proof that** $\phi \in$3SAT **iff** $\langle G \rangle \in$Partition-into-Triangles:

$\Rightarrow$) Suppose that $\langle \phi \rangle \in$3SAT:

Then there exists some truth assignment that assigns at least one literal per clause to true.

If we use that truth assignment to choose an assignment to triangles for the variable widgets, these literals will be free to be assigned with the $n$ clause widgets.

This will assign all of the inner nodes from the variable widgets to triangles, and will also assign all of the nodes from clause widgets to triangles. But it will leave $(k-1)n$ of the outer nodes from the variable widgets to be assigned.

By assigning these nodes to the extra widgets from end of the construction.

This will give us a decomposition of $G$ into triangles.

$\Rightarrow \langle G \rangle \in$Partition-into-Triangles.

$\Leftarrow$) Suppose that $\langle G \rangle \in$Partition-into-Triangles:

Then There is some way of breaking $G$ into disjoint triangles.

In this partitioning, each of the clause widgets must contribute to exactly one triangle.

The extra vertex in each of these triangles must correspond to some literal on the outside of the variable widgets.

Since the variable widget enforces consistency in the variable assignments, the literals chosen by the clause widgets are consistent.

$\Rightarrow$ These literal assignments can be extended into a consistent truth assignment.

Since this truth assignment sets every clause to true, it is a satisfying assignment for $\phi$.

$\Rightarrow \langle\phi\rangle \in$3SAT.

Finally, we can see that the number of nodes in $G$ is $3kn$, and that we can tell in polytime whether any pair of vertices is an edge. So the entire construction takes polynomial time. So 3SAT $\leqslant_p$ PARTITION-INTO-TRIANGLES, and so PARTITION-INTO-TRIANGLES is NP-complete.

■

---

## 3.11 3DM

Definition of 3DM:

**Instance:** Three sets $A$, $B$, and $C$, where $|A| = |B| = |C|$, and a set $T \subseteq A \times B \times C$.

**Question:** Is there a subset $M \subseteq T$ such that:

i) $|M| = |A|$,

ii) For any distinct $(a, b, c)$ and $(a', b', c') \in M, a \neq a', b \neq b'$, and $c \neq c'$?

**Reduced from:** 3SAT.

Firstly, we can see that this is in NP:

1) It is a yes/no problem.
2) Its certificate is the set $M$.
3) Since $M \subseteq T$, and $T$ is part of the input, $M$ is polynomial in the size of the input.
4) A verifier for this certificate would be:

   Let $\mathcal{V}$ =“On $\langle A, B, C, T, M\rangle$:”

   1. Reject if any of the following checks fails:
      $\mathcal{O}(1)$ *time.*
   2. Check that $M \subseteq T$.
      $\mathcal{O}(|A|^6)$ *time.*
   3. Check that $|M| = |A|$.
      $\mathcal{O}(|A|)$ *time.*
   4. Check that for any distinct $(a, b, c)$ and $(a', b', c') \in M, a \neq a', b \neq b'$, and $c \neq c'$.
      $\mathcal{O}(|A|^2)$ *iterations.*
   5. *Accept.*
      $\mathcal{O}(1)$ *time.*

5) We can see that this verifier runs in $\mathcal{O}(|A|^6)$ time, and so is polytime in the size of $\langle A, B, C, T\rangle$.

So 3DM is in NP.

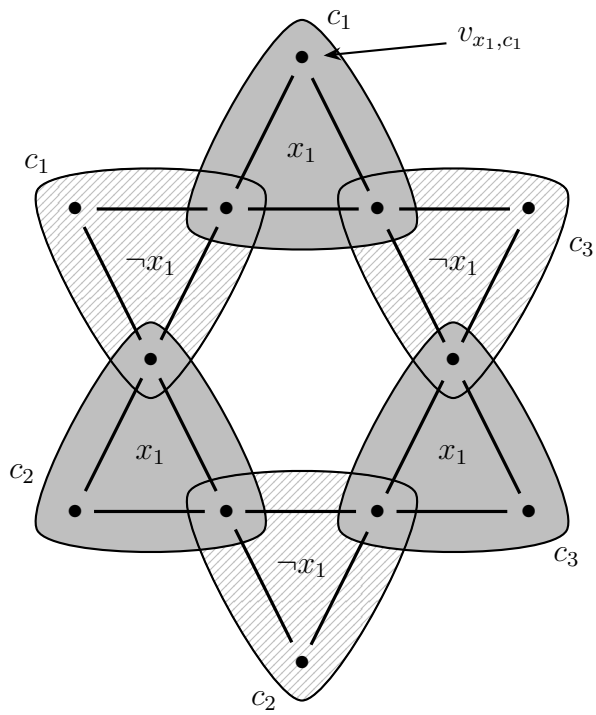
*We'll use an interesting trick for this reduction: Note that the* 3DM *problem is closely related to the* PARTITION-INTO-TRIANGLES *problem, in that they both relate to partitioning a set into pre-defined triples. In principle we'd like to reduce from one to the other, but that's not so easy to do here – it's not clear how to split the vertices in a graph $G$ into the sets $A$, $B$, and $C$, but that's what we'd need to do for a reduction from* PARTITION-INTO-TRIANGLES *into* 3DM.

*On the other hand, we can easily split up the vertices in this way for the specific type of graph we get from the reduction into* PARTITION-INTO-TRIANGLES. *This means that we can use essentially the same reduction for each problem. In fact, we build the same graph $G$ in both reductions – the difference is that in the* PARTITION-INTO-TRIANGLES *reduction we directly return the graph $G$, but for the* 3DM *problem we label the vertices of $G$, and return the sets of labels for the $A$, $B$, and $C$. The $T$ is just a list of the triangles in $G$. So the process is just a matter of finding the right labels for the vertices in the* PARTITION-INTO-TRIANGLES *reduction graph $G$.*

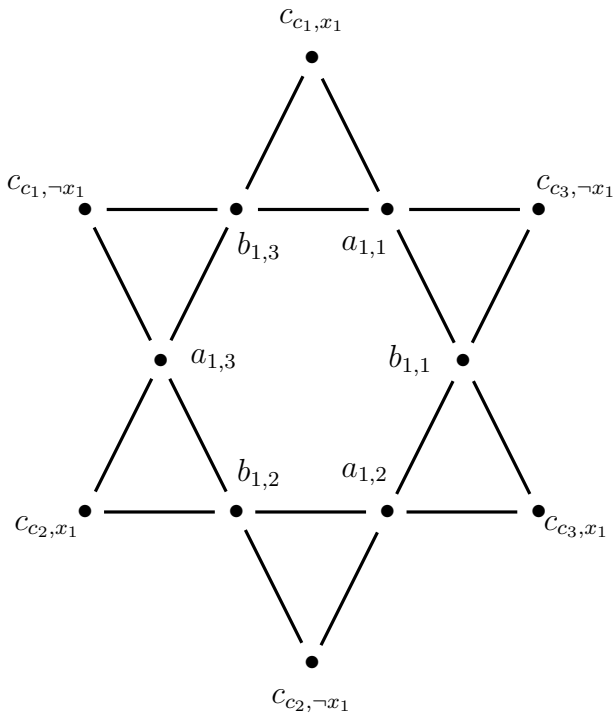
Suppose we have an input 3CNF $\phi$. We'll assume using the lemma in section 2 that this $\phi$ has three distinct variables per clause (i.e., no $(x \vee x \vee y)$ type clauses).

Now, consider the PARTITION-INTO-TRIANGLES reduction we just finished. Let $G$ be the graph we get from that reduction. Now, $G$ is made out of variable widgets, clause widgets, and garbage collection widgets.

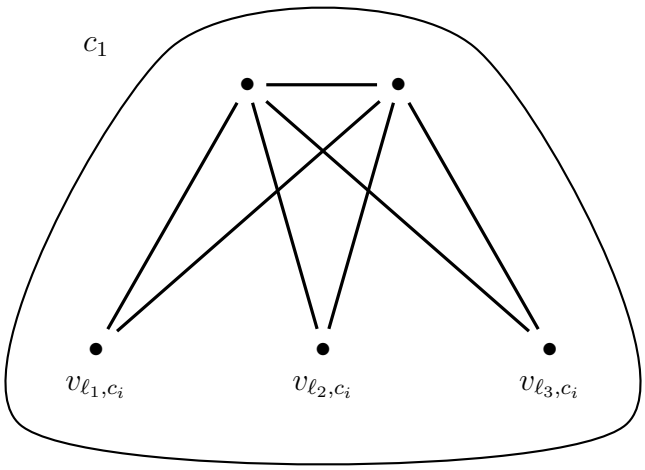
Recall that the variable widgets will be built with one widget per variable, and each widget will be a star with $2n$ points, where $n$ is the number of clauses in $\phi$. So for a 3-clause $\phi$, the widget for $x_1$ would be of the form:



What we want to do is label each of the vertices. So we'll give the vertices these labels:

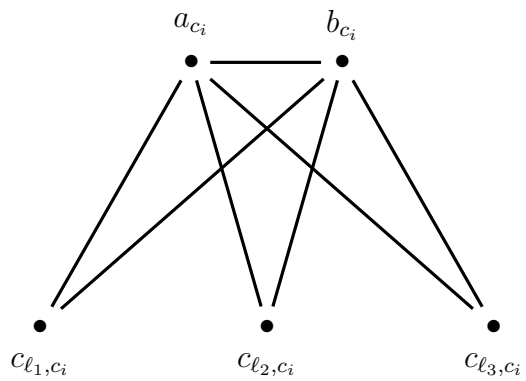



Recall that the variable $v_{x_1,c_1}$ is a variable that remains free if we choose $x_1 = T$, while the other variable is taken. The same applies for the other clauses and variables. With this in mind, recall that in the PARTITION-INTO-TRIANGLES reduction, for the clause $c_i = (\ell_{i_1} \vee \ell_{i_2} \vee \ell_{i_3})$, where $\ell_{i_1}, \ell_{i_2}$, and $\ell_{i_3}$ are the literals in $c_i$, we produced the following widget:




So now we want to label these nodes. The labels we give are as follows:

> *Remember that the vertices on the bottom row are also part of the variable widgets, so they're already labelled!*




Finally, we have to label the nodes from the garbage collection widgets. But the garbage collection widgets are just $k(n-1)$ copies of the clause widgets that connect to every $v_{x_i,c_j}$ and $v_{\neg x_i,c_j}$ node, and so we label them in the same way.

If we assign all of the $a$ labels to the set $A$, the $b$ labels to the set $B$, and the $c$ labels to the set $C$, then we can see that $|A| = |B| = |C| = 2nk$, where $n$ is the number of clauses in $\phi$ and $k$ is the number of variables. Furthermore, every triangle in $G$ contains exactly one of each type of label (has a triple of labels in $A \times B \times C$). So we can set $T$ to be a list of all of the triangles in $G$.

*Note that we can find $T$ in polynomial time by looping over all triples of vertices!*

Clearly, any partitioning of $G$ into disjoint triangles will induce a subset $M \subseteq T$ that satisfies properties i) and ii) in the 3DM problem defintion, and vice-versa. So $\langle G \rangle$ is in PARTITION-INTO-TRIANGLES iff $\langle A, B, C, T \rangle$ is in 3DM, and therefore $\langle A, B, C, T \rangle \in$3DM iff $\langle \phi \rangle \in$ 3SAT.

*We refer the reader to the* PARTITION-INTO-TRIANGLES *reduction for the argument that $G$ can be partitioned into triangles iff $\phi$ has a satisfying truth assignment.*

Finally, we can see that this reduction is polytime: the construction of $G$ is polytime in the size of $\phi$, as described in the PARTITION-INTO-TRIANGLES reduction. The label sets $A$, $B$, and $C$ are simply labels of the vertices of $G$, and can be efficiently computed. Finally, the set $T$ can be found in polytime by looping over the triples of vertices in $G$, and so can also be found in polytime. So the entire construction is polytime, as required.

■

---

## 3.12 SUBSET-SUM

Definition of SUBSET-SUM:

> **Input:** A multiset $S$ of non-negative integers, an integer $t$.
>
> **Question:** Is there a subset $S' \subseteq S$ such that $\sum_{x \in S'} = t$?

**Reduced from:** 3SAT.

*Note: This problem is still* NP*-complete even if $S$ is a proper set, as opposed to a multiset (i.e., it has no repititions). Our reduction here doesn't prove this (it outputs a multiset), but you've probably written a reduction that does output a set in your week 8 tutorial.*
Proof that SUBSET-SUM is in NP:

1) It is a yes/no problem.

2) A certificate would be the subset $S'$.

3) Since $S' \subseteq S$, and since $S$ is listed in the input, the certificate is polynomial in the size of the input.

4) The verifier would be:

   $\mathcal{V}$ = "On input $\langle S, t, S' \rangle$:

   1. Check that $S' \subseteq S$. If it is not, *reject.*

      $\mathcal{O}(|\langle S \rangle|^2)$ *time, where* $|\langle S \rangle|$ *is the space need to write the elements of* $S$.

   2. Let $x = 0$.

      $\mathcal{O}(1)$ *time.*

   3. For $y \in S'$:

      $\mathcal{O}(|\langle S \rangle|)$ *iterations.*

   4. $\quad x+ = y$.

      $\mathcal{O}(|\langle S \rangle|)$ *time per iteration.*

   5. If $x == t$, *accept*, otherwise *reject.*

      $\mathcal{O}(|\langle S \rangle| + \log t)$ *time.*

5) We can see that this verifier runs in $\mathcal{O}(|\langle S \rangle|^2 + \log t)$ time, and so is polytime in the size of $\langle S, t \rangle$.

Proof that 3SAT $\leqslant_p$ SUBSET-SUM:

Given an input 3CNF $\phi$, we'll write the variables of $\phi$ as $x_1, \ldots, x_\ell$ and its clauses as $c_1, \ldots, c_k$. Note that we can assume using the lemma in section 2 that the clauses in $\phi$ each have three distinct variables.

**Idea:** Let's start by setting up a table like so:

| | 1 | 2 | 3 | ... | $\ell$ | |
|---|---|---|---|---|---|---|
| $x_1$ | 1 | 0 | 0 | ... | 0 | $\langle$ something $\rangle$ |
| $\neg x_1$ | 1 | 0 | 0 | ... | 0 | $\langle$ something $\rangle$ |
| $x_2$ | 0 | 1 | 0 | ... | 0 | $\langle$ something $\rangle$ |
| $\neg x_2$ | 0 | 1 | 0 | ... | 0 | $\langle$ something $\rangle$ |
| $x_3$ | 0 | 0 | 1 | ... | 0 | $\langle$ something $\rangle$ |
| $\neg x_3$ | 0 | 0 | 1 | ... | 0 | $\langle$ something $\rangle$ |
| $\vdots$ | | | | $\vdots$ | | $\vdots$ |
| $x_\ell$ | 0 | 0 | 0 | ... | 1 | $\langle$ something $\rangle$ |
| $\neg x_\ell$ | 0 | 0 | 0 | ... | 1 | $\langle$ something $\rangle$ |
| | 1 | 1 | 1 | ... | 1 | $\langle$ something $\rangle$ |

*So two rows per variable.*

Suppose that we want to choose a set of rows whose columns add up to the numbers on the bottom. Then, for any $x_i$, we would have to choose either the row headed by $x_i$ or the one headed by $\overline{x_i}$. We wouldn't be able to choose both.

*This is a bit like forcing every boolean variable $x_i$ to be either true or false - the rows we'd choose above correspond to truth assignments!*

Let's continue with this idea: can we tell if a clause is satisfied by a truth assignment? We've left some columns in the table, so let's use them. Suppose, for example, that $c_1$ is $(x_1 \vee \neg x_2 \vee x_3)$. We can encode this in the first column on the right like so:

| | 1 | 2 | 3 | ... | $\ell$ | $c_1$ | ... |
|---|---|---|---|---|---|---|---|
| $x_1$ | 1 | 0 | 0 | ... | 0 | 1 | ... |
| $\neg x_1$ | 1 | 0 | 0 | ... | 0 | 0 | ... |
| $x_2$ | 0 | 1 | 0 | ... | 0 | 0 | ... |
| $\neg x_2$ | 0 | 1 | 0 | ... | 0 | 1 | ... |
| $x_3$ | 0 | 0 | 1 | ... | 0 | 1 | ... |
| $\neg x_3$ | 0 | 0 | 1 | ... | 0 | 0 | ... |
| $\vdots$ | | | | $\vdots$ | | | $\vdots$ |
| $x_\ell$ | 0 | 0 | 0 | ... | 1 | 0 | ... |
| $\neg x_\ell$ | 0 | 0 | 0 | ... | 1 | 0 | ... |
| | 1 | 1 | 1 | ... | 1 | ? | ... |

That is, we've added a 1 in this column to every row corresponding to a literal in $c_1$.

Clearly, the sum of the $c_1$ column is between 1 and 3 if we choose a truth assignment that satisfies $c_1$.

What's more, we can repeat the same process for every clause:

| | 1 | 2 | 3 | ... | $\ell$ | $c_1$ | $c_2$ | ... | $c_k$ |
|---|---|---|---|---|---|---|---|---|---|
| $x_1$ | 1 | 0 | 0 | ... | 0 | 1 | 0 | ... | 0 |
| $\neg x_1$ | 1 | 0 | 0 | ... | 0 | 0 | 0 | ... | 0 |
| $x_2$ | 0 | 1 | 0 | ... | 0 | 0 | 1 | ... | 0 |
| $\neg x_2$ | 0 | 1 | 0 | ... | 0 | 1 | 0 | ... | 0 |
| $x_3$ | 0 | 0 | 1 | ... | 0 | 1 | 0 | ... | 0 |
| $\neg x_3$ | 0 | 0 | 1 | ... | 0 | 0 | 1 | ... | 1 |
| $\vdots$ | | | | $\vdots$ | | | | $\vdots$ | |
| $x_\ell$ | 0 | 0 | 0 | ... | 1 | 0 | 0 | ... | 1 |
| $\neg x_\ell$ | 0 | 0 | 0 | ... | 1 | 0 | 0 | ... | 0 |
| | 1 | 1 | 1 | ... | 1 | ? | ? | ... | ? |

So we've made a table where any satisfying truth assignment to $\phi$ gives us a 1 in the first set of columns and something between 1 and 3 in the second set of columns. Moreover, if there is no satisfying assignment we'll never be able to satisfy every clause, so at least one clause from the second column will always add up to 0.

If we add a couple of dummy variables to every clause column, we can make them add up to 3 when there's a satisfying assignment:

| | 1 | 2 | 3 | ... | $\ell$ | $c_1$ | $c_2$ | ... | $c_k$ |
|---|---|---|---|---|---|---|---|---|---|
| $x_1$ | 1 | 0 | 0 | ... | 0 | 1 | 0 | ... | 0 |
| $\neg x_1$ | 1 | 0 | 0 | ... | 0 | 0 | 0 | ... | 0 |
| $x_2$ | 0 | 1 | 0 | ... | 0 | 0 | 1 | ... | 0 |
| $\neg x_2$ | 0 | 1 | 0 | ... | 0 | 1 | 0 | ... | 0 |
| $x_3$ | 0 | 0 | 1 | ... | 0 | 1 | 0 | ... | 0 |
| $\neg x_3$ | 0 | 0 | 1 | ... | 0 | 0 | 1 | ... | 1 |
| $\vdots$ | | | | $\vdots$ | | | | $\vdots$ | |
| $x_\ell$ | 0 | 0 | 0 | ... | 1 | 0 | 0 | ... | 1 |
| $\neg x_\ell$ | 0 | 0 | 0 | ... | 1 | 0 | 0 | ... | 0 |
| $d_1$ | | | | | | 1 | 0 | ... | 0 |
| $d'_1$ | | | | | | 1 | 0 | ... | 0 |
| $d_2$ | | | | | | 0 | 1 | ... | 0 |
| $d'_2$ | | | | | | 0 | 1 | ... | 0 |
| $\vdots$ | | | | | | | | $\vdots$ | |
| $d_k$ | | | | | | 0 | 0 | ... | 1 |
| $d'_k$ | | | | | | 0 | 0 | ... | 1 |
| $t$ | 1 | 1 | 1 | ... | 1 | 3 | 3 | ... | 3 |

So we can choose a set of rows in this table whose columns add up to the bottom row iff $\phi$ has a satisfying assignment.

To turn this into a SUBSET-SUM instance, just note that the numbers in the columns can never add up to more than 3 – so we can treat the rows as numbers in base 10 and never have to worry about one digit affecting another. So our $S$ will just be the non-bottom rows in the table, and $t$ will be the bottom row.

We can see that this construction will take polynomial time:

> The total table size is $(\ell + k)^2$, which is clearly polynomial in the size of $\phi$. Each entry in the table is a 0, a 1, or a 3, and each value can be found with at worst a polynomial time lookup. The string value of $t$, $\langle t \rangle$ is $1^\ell 3^k$, which can also be found in polynomial time. So all told, this construction takes polynomial time.

And we've already given an overview of our justification as to why it works.

So 3SAT $\leqslant_p$ SUBSET-SUM, and so SUBSET-SUM is NP-complete.

■

---

## 3.13 JOB-SEQUENCING

Definition of JOB-SEQUENCING:

> **INPUT:** An execution time vetor $(T_1, T_2, \ldots, T_p) \in \mathbb{Z}^p$, a deadline vetor $(D_1, D_2, \ldots, D_p) \in \mathbb{Z}^p$, a penalty vetor $(P_1, P_2, \ldots, P_p) \in \mathbb{Z}^p$, and a number $k \in \mathbb{N}$.

**QUESTION:** Is there a permutation $\pi$ of $\{1, 2, \ldots, p\}$ (i.e., an ordering of the jobs) such that if

$$\delta_{\pi(i)} = \begin{cases} 1, & \sum_{j=1}^{i} T_{\pi(j)} > D_{\pi(i)} \\ 0, & \text{otherwise,} \end{cases}$$

then

$$\sum_{i=1}^{p} \delta_{\pi(i)} P_{\pi(i)} \leqslant k?$$

*Basically, what we're doing here is this: we have $p$ jobs, numbered 1 to $p$. Each job $i$ has a time $T_i$ that it takes to run, and a deadline $D_i$. If the job is not done by that deadline, we pay a penalty $P_i$.*

*So what we want to do is find an ordering $\pi$ of the jobs $\{1, \ldots, p\}$ that minimizes the penalty we pay (or at least makes it no more than $k$). That ordering is a list of the numbers 1 to $p$ (not necessarily on order) that tells us which jobs goes when: e.g., if we have three jobs 1, 2, 3, then the ordering $\pi = [3, 1, 2]$ would tell us that job 3 goes first, then job 1, then job 2. The first job in this ordering would be $\pi(1) = 3$.*

**Reduced from:** SUBSET-SUM.

Proof that JOB-SEQUENCING is in NP:

1) This is a decision problem.

2) A certificate for an instance $\langle \overline{T}, \overline{D}, \overline{P} \rangle$ of this problem would be the ordering $\pi$. We'll treat this is a list of numbers from 1 to $p$.

3) Since this is a list of the numbers 1 to $p$, and since the three vectors in the input are of length $p$, this certificate is polysize in the size of the input.

4) Our verifier will be:

$\mathcal{V}$ = "On input $\langle \overline{T}, \overline{D}, \overline{P}, k, \pi \rangle$:

1. Reject if any of the following checks fails. $\mathcal{O}(1)$ *time.*
2. Check that $|\pi| = p$. $\mathcal{O}(p)$ *time.*
3. Check that each number from 1 to $p$ occurs exactly once in $\pi$. $\mathcal{O}(p^2)$ *time.*
4. *penalty* $= 0$. $\mathcal{O}(1)$ *time.*
5. For $i = 1$ to $p$:

   $\mathcal{O}(p)$ *iterations.*
6. *count* $= 0$. $\mathcal{O}(1)$ *time per iteration.*
7. For $j = 1$ to $i$:

   $\mathcal{O}(p)$ *iterations.*
8. *count* $+= T_{\pi(j)}$. $\mathcal{O}(\log_{pT_{\pi(j)}})$ *time.*
9. If *count* $> D_{\pi_i}$: $\mathcal{O}(\log_{D_{\pi(i)}})$ *time per iteration.*

10. *penalty* $+= P_{\pi(i)}$ $\mathcal{O}(\log_{pP_{\pi(i)}})$ *time per iteration.*
11. Check that *penalty* $\leqslant k$. $\mathcal{O}(\log_k)$ *time.*
12. *Accept.* $\mathcal{O}(1)$ *time.*"

5) We can see that this algorithm takes $\mathcal{O}(p^2)$ steps outside of the main loops, and that it takes $\mathcal{O}(p^2)$ loops each loop takes a time polynomial in the size of the input (being additions and comparisons of numbers in the input), and so the overall algorithm takes polytime.

Proof that SUBSET-SUM $\leqslant_p$ JOB-SEQUENCING:

Let $\langle S, t\rangle$, be a SUBSET-SUM instnance, where $S = \{s_1, s_2, \ldots, s_n\}$ is a finite multiset of non-negative integers.

We set $p = n$. For each $1 \leqslant i \leqslant p, T_i = P_i = s_i$, and $D_i = t$. We set $k = \left[\sum_{s_i} \in S\right] - t$.

We argue that $\langle \overline{T}, \overline{D}, \overline{P}, k\rangle \in$JOB-SEQUENCING iff $\langle S, t\rangle \in$ SUBSET-SUM:

Suppose $\langle S, t\rangle \in$ SUBSET-SUM.

Then $S$ has a subset $X$ that sums to $t$.

Let $i' = |X|$, and let $\pi$ be the ordering that lists every element of $X$ (in order), followed by every element of $S \setminus X$ (in order).

Now, for each $i, 1 \leqslant i \leqslant i'$, we can see that

$$\begin{aligned}
\sum_{j=1}^{i} T_{\pi(j)} &\leqslant \sum_{j=1}^{i'} T_{\pi(j)} \\
&= \sum_{j=1}^{i'} s_{\pi(j)} \\
&= \sum_{s_i \in X} s_i \\
&= t \\
&= D_{\pi(i)}
\end{aligned}$$

And so $\delta_{\pi(i)} = 0$. Thus,

$$\begin{aligned}
\sum_{i=1}^{p} \delta_{\pi(i)} P_{\pi(i)} &\leqslant \sum_{i=i'+1}^{p} P_{\pi(i)} \qquad \textit{(This is actually an equality.)} \\
&= \sum_{i=i'+1}^{p} s_{\pi(i)} \\
&= \sum_{s_i \in S \setminus X} s_{\pi(i)} \\
&= \left[\sum_{s_i \in S} s_i\right] - t \\
&= k.
\end{aligned}$$

$\Rightarrow \langle \overline{T}, \overline{D}, \overline{P}, k \rangle \in$JOB-SEQUENCING.

Suppose $\langle \overline{T}, \overline{D}, \overline{P}, k \rangle \in$JOB-SEQUENCING.

Then there is some $\pi$ such that $\sum_{i=1}^{p} \delta_{\pi(i)} P_{\pi(i)} \leqslant k = \big[ \sum_{s_i \in S} s_i \big] - t$.

Let $X$ be the set $\big\{ s_{\pi(i)} \,\big|\, \delta_{\pi(i)} = 0 \big\}$, and let $i' = |X|$.

By our definition of $\delta_{\pi(i)}$, we can see that for any $i, 1 \leqslant i \leqslant i'$, $\sum_{j=1}^{i} T_{\pi(j)} \leqslant D_{\pi(i)} = t$.

In particular, $\sum_{s_{\pi(i)} \in X} T_{\pi(i)} = \sum_{j=1}^{i'} T_{\pi(j)} \leqslant t$.

On the other hand,

$$\begin{aligned}
\sum_{s_{\pi(i)} \in S \setminus X} T_{\pi(i)} &= \sum_{s_{\pi(i)} \in S \setminus X} \delta_{\pi(i)} T_{\pi(i)} \\
&\textit{Recall that } s_{\pi(i)} \in X \leftrightarrow \delta_{\pi(i)} = 0. \\
&\leqslant \big[ \sum_{s_i \in S} s_i \big] - t
\end{aligned}$$

And so

$$\begin{aligned}
\sum_{s_i \in S} s_i &= \sum_{s_i \in S} T_i \\
&= \sum_{s_{\pi(i)} \in S} T_{\pi(i)} \\
&= \sum_{s_{\pi(i)} \in X} T_{\pi(i)} + \sum_{s_{\pi(i)} \in S \setminus X} T_{\pi(i)} \\
&\leqslant t + \big[ \sum_{s_i \in S} s_i \big] - t \\
&= \sum_{s_i \in S} s_i.
\end{aligned}$$

And so this is an equality.

Thus, we can see that $\sum_{s_{\pi(i)} \in X} T_{\pi(i)} = \sum_{s_{\pi(i)} \in X} s_{\pi(i)} = \sum_{s_i \in X} s_i = t$.

$\Rightarrow \langle S, t \rangle \in$ SUBSET-SUM.

Finally, we can see that this reduction consists of copying out $S$ twice, of copying the value $t$ $n$ times, and of adding up the values in $S$ (and subtracting $k$). All of these operations take polynomial time, and so the entire reduction is polytime.

So JOB-SEQUENCING is NP-complete, as required.

■

## 3.14 TIMETABLE-SCHEDULING

Definition of TIMETABLE-SCHEDULING *Problem and reduction from (Even et al., 1975), via (Garey and Johnson, 1979).*:

**Instance:** We are given the following sets:

- A finite set $H$ of available hours $\{1, \ldots, k\}$.
- A series $\mathcal{T}$ of subsets $\{T_1, \ldots T_n\}$, where each $T_i \subseteq H$.

  *In our problem, there are $n$ teachers available. The set $T_i$ is the set of hours in which teacher $i$ is available to teach.*
- A series $\mathcal{C}$ of subsets $\{C_1, \ldots C_m\}$, where each $C_j \subseteq H$.

  *In our problem, there are $m$ classes to be taught. The set $C_j$ is the set of hours in which class $j$ can be taught.*
- An $n \times m$ matrix $R$ of non-negative integers.

  *In our problem, the entry $r_{ij}$ of $R$ is the number of hours that teacher $i$ is required to teach class $j$.*

**Question:** Is there a possible timetable $f(i, j, h)$ for the teachers and classes that satisfies the following constraints?

a) $f(i, j, h) = 1 \rightarrow h \in T_i \cap C_j$,

   *The function $f(i, j, h)$ takes the values 0 and 1, and it takes the value 1 iff teacher $i$ is assigned to class $j$ during the hour $h$. This constraint ensures that we only assign teachers to classes in times that both the teacher and class are avaiable.*

b) $\sum_{h=1}^{k} f(i, j, h) = r_{ij}$ for all $1 \leqslant i \leqslant n$ and $1 \leqslant j \leqslant m$.

   *This constraint ensures that teacher $i$ is assigned all of the hours required to teach the class $j$.*

c) $\sum_{i=1}^{n} f(i, j, h) \leqslant 1$ for all $1 \leqslant j \leqslant m$ and $1 \leqslant h \leqslant k$.

   *This constraint ensures that at most one teacher is assigned to teach class $j$ in any one time slot.*

d) $\sum_{j=1}^{m} f(i, j, h) \leqslant 1$ for all $1 \leqslant i \leqslant n$ and $1 \leqslant h \leqslant k$.

   *This constraint ensures that no teacher is assigned more than one class in any one time slot.*

**Reduced from:**3SAT.

Firstly, we can see that this is in NP:

1) It is a yes/no problem.
2) Its certificate is the timetable function $f$. We'll treat this as an $n \times m \times k$ table.

3) The size of $f$ is $\mathcal{O}(i \times j \times k)$. The input includes lists of length $i$, $j$, and $k$, and so this certificate is polynomial in the size of the input.

4) A verifier for this certificate would be:

Let $\mathcal{V}$ ="On $\langle H, \mathcal{T}, \mathcal{C}, R, f\rangle$:"

1. Check that $f$ is a function from $\mathcal{T} \times \mathcal{C} \times H$ to $\{0, 1\}$.
   $\mathcal{O}(nmk)$ *time.*
2. Check that $f(i, j, h) = 1$ only if $h \in T_i \cap C_j$.
   $\mathcal{O}(nmk[n + m])$ *time.*

   *Here we're treating the table lookups fpor $f$ (and $R$) as constant time, but searching through $T_i$ and $C_j$ as linear. We can do a bit better in the searches, and are likely a bit worse in the lookups, but this is an upper bound to within a logarithmic factor.*

3. Check that for all $i$ and $j$, $\sum_{h=1}^{k} f(i, j, h) = r_{ij}$.
   $\mathcal{O}(nmk)$ *time.*

   *Recall that the addition of $k$ 1s amortizes to $\mathcal{O}(1)$ time, even if general inteeger additon isn't necessarily constant time.*

4. Check that for all $j$ and $h$, $\sum_{i=1}^{n} f(i, j, h) \leqslant 1$.
   $\mathcal{O}(nmk)$ *time.*
5. Check that for all $i$ and $h$, $\sum_{j=1}^{n} f(i, j, h) \leqslant 1$.
   $\mathcal{O}(nmk)$ *time.*
6. If all of these checks pass, *accept*, otherwise *reject.*
   $\mathcal{O}(1)$ *time.*

5) We can see that this verifier runs in $\mathcal{O}(nmk[n + m])$ time, and so is polytime in the size of the input.

So TIMETABLE-SCHEDULING is in NP.

**Reduction from 3SAT**:

Let an instance $\langle\phi\rangle$ of 3SAT be given, where $\phi$ has $n$ clauses and $k$ variables. Note that we can assume using the lemma in section 2 that the clauses in $\phi$ each have three distinct variables.

To start, we set $H = \{1, 2, 3\}$. We'll use the classes to represent whether a variable $x_i$ occurs in clause $D_j$ of $\phi$, with the understanding that we'll add extra classes to help enforce consistency. We'll use the teachers to indicate our truth assignments, also with the understanding that we'll add extra teachers to help enforce consistency. Every class will be avaiable on all three hours (i.e., $C_j = H$ for all $j$).

Our teachers will be kept on a *tight* schedule in the sense that for all $i$, $\sum_{j=1}^{m} r_{ij} = |T_i|$. That is, every teacher will have to teach whenever avaiable. Also, every teacher will be required to teach either two or three hours.

To (hopefully) help make our reduction more clear, we'll show the classes and times graphically, with the columns of a grid representing our classes, and the rows representing the times:

| | | | | | |
|---|---|---|---|---|---|
| $h = 1$ | | | | | |
| $h = 2$ | | | | | |
| $h = 3$ | | | | | |
| | $C_{j_1}$ | $C_{j_2}$ | $C_{j_3}$ | $C_{j_4}$ | $C_{j_5}$ |

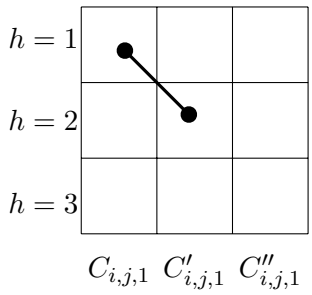
Now, for any variable $x_i$, let $p(i)$ be the number of clauses of $\phi$ in which either $x_i$ or $\neg x_i$ occurs. For each of these variables we will create $5p(i)$ classes, which we will label as $C_{i,1,1}, C_{i,1,2}, C_{i,1,3}, C_{i,1,4}, C_{i,1,5}, C_{i,2,1}, \ldots, C_{i,p(i),5}$:

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| $h = 1$ | | | | | | | | |
| $h = 2$ | | | | | | | | |
| $h = 3$ | | | | | | | | |
| | $C_{i,1,1}$ | $C_{i,1,2}$ | $C_{i,1,3}$ | $C_{i,1,4}$ | $C_{i,1,5}$ | $C_{i,2,1}$ | $\cdots$ | $C_{i,p(i),5}$ |

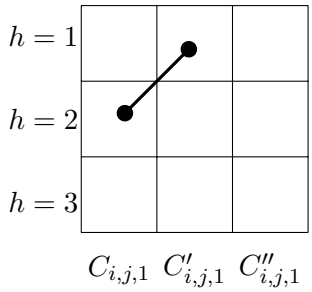
Now, we'll want to add some restrictions to which of our time slots are available. We'll do so by adding two copies of each class $C_{i,j,1}$:

| | | | |
|---|---|---|---|
| $h = 1$ | | | |
| $h = 2$ | | | |
| $h = 3$ | | | |
| | $C_{i,j,1}$ | $C'_{i,j,1}$ | $C''_{i,j,1}$ |

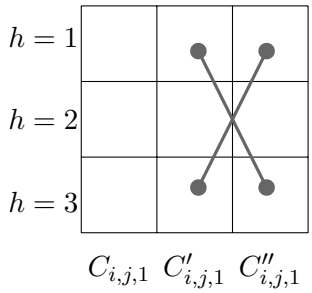
We'll add a teacher $T_{i,j,1}$ who is available in hours $h = 1$ and $h = 2$, and who must teach one hour each of $C_{i,j,1}$ and $C'_{i,j,1}$. This teacher has two possible schedules:




or

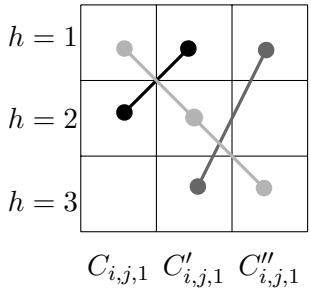



We'll add a second teacher $T'_{i,j,1}$ who is available in time slots $h = 1$ and $h = 3$, and who must teach classes $C'_{i,j,1}$ and $C''_{i,j,1}$. So we have one of the two following possible schedules for this teacher (we'll write both as lines on the grid):

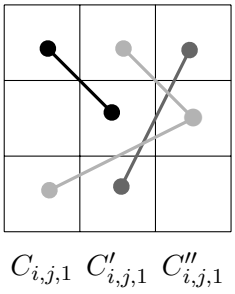



Finally, we'll add a third teacher $T''_{i,j,1}$ who must teach all three of these classes, and is avaiable in all three time slots. We can see that this gives us four possible schedules:

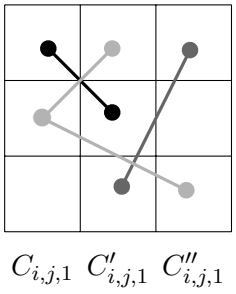
either



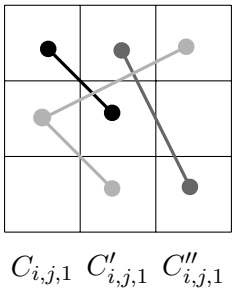

or




or

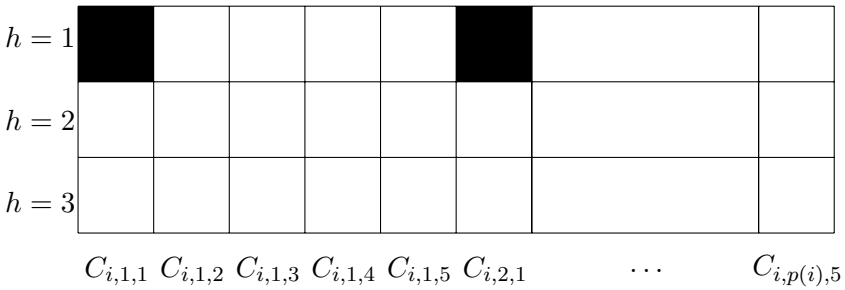



or



In any of these cases, we see that class $C_{i,j,1}$ is taken in time slot $h = 1$. One of its other two time slots is also taken, but we get to choose which one.

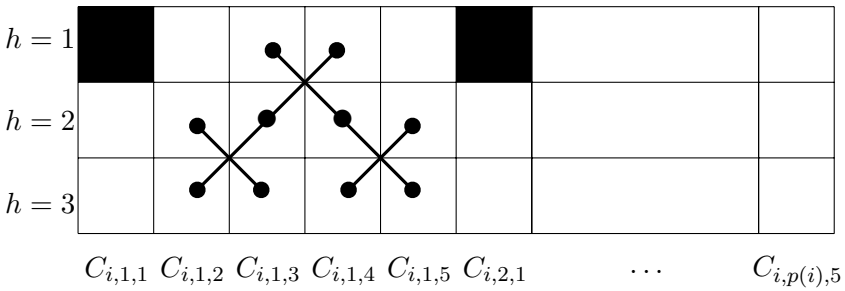
With this in mind, we'll remove the time slot $h = 1$ from our timetable for the classes $C_{i,j,1}$:




For each $1 \leqslant j \leqslant p(i)$, we'll add three teachers: $T_{i,j,3/4}$, $T_{i,j,2/3}$, and $T_{i,j,4/5}$ each of whom will teach exactly two hours:

- The first, $T_{i,j,3/4}$, will be available in time slots $h = 1$ and 2, and will teach the classes $C_{i,j,3}$ and $C_{i,j,4}$.
- The second, $T_{i,j,2/3}$, will be available in time slots $h = 2$ and 3, and will teach the classes $C_{i,j,2}$ and $C_{i,j,3}$.
- The third, $T_{i,j,4/5}$, will be available in time slots $h = 2$ and 3, and will teach the classes $C_{i,j,4}$ and $C_{i,j,5}$.

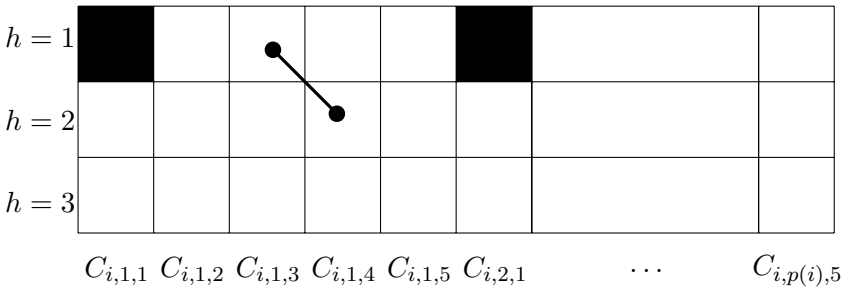
So the time slots used for $j = 1$ will be some combination of:



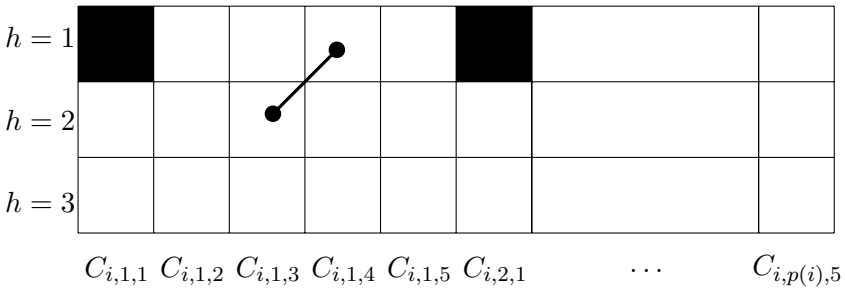

Now, we can see that there are two ways of setting the assignment of the teacher

$$T_{i,j,3/4}$$

:



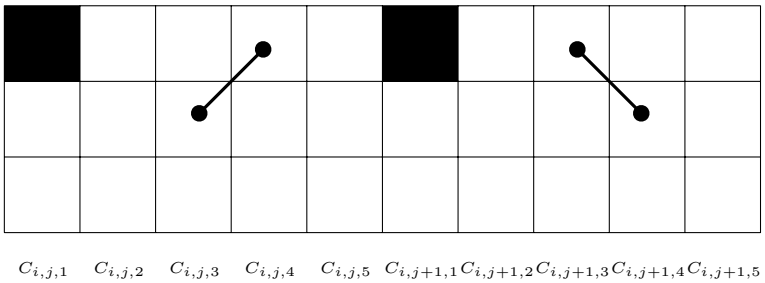
or




We can force this choice to be consistent across all $1 \leqslant j \leqslant i$ for this $i$ by adding a fourth teacher $T_{i,j,c}$ for each $j$: This teacher will be avaiable for all three time slots, and will teach the classes $C_{i,j,4}$, $C_{i,j+1,1}$, and $C_{i,j+1,3}$.

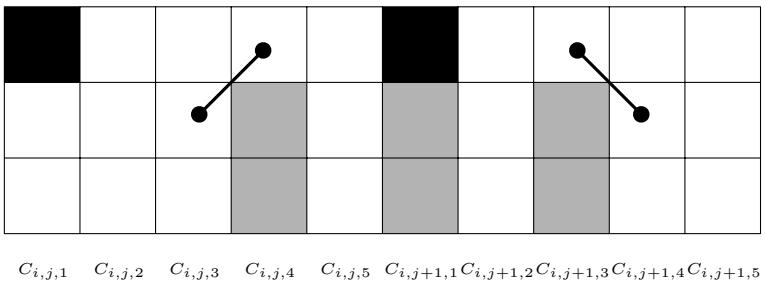
> *Note that we wrap around when we hit the end of the table: if $j = p(i)$, then the index referred to by $j + 1$ is just 1 – i.e., our additions are modulo $p(i)$.*

The reason this forces consistency is as follows: if the teachers $T_{i,j,3/4}$ are not assigned the classes $C_{i,j,3}$ consistently, then there must be some point for which their assignments are:



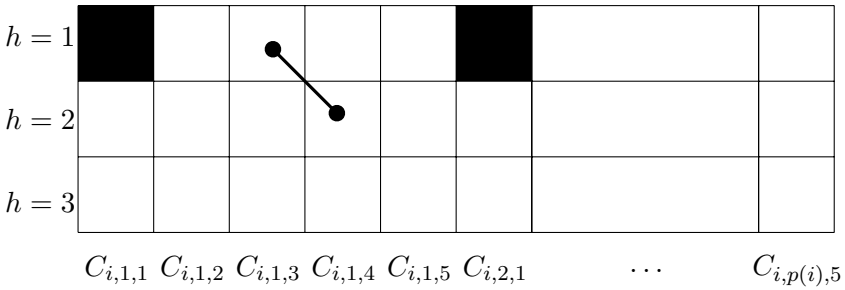

> *Note again that we wrap this table around at its end point – we may have $j = p(i)$ and $j + 1 = 1$!*

If this is the case we have to assign our new teacher $T_{i,j,c}$ to the classes $C_{i,j,4}$, $C_{i,j+1,1}$, and $C_{i,j+1,3}$. The time slots available to us for our assignment would be the gray squares in the following timetable:

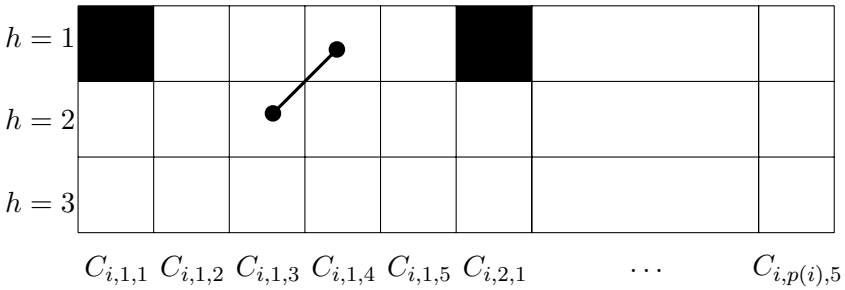



We would have to assign this teacher to three classes, but only have two available timeslots to do it in. We can see by the pigeonhole principle that this is not possible.

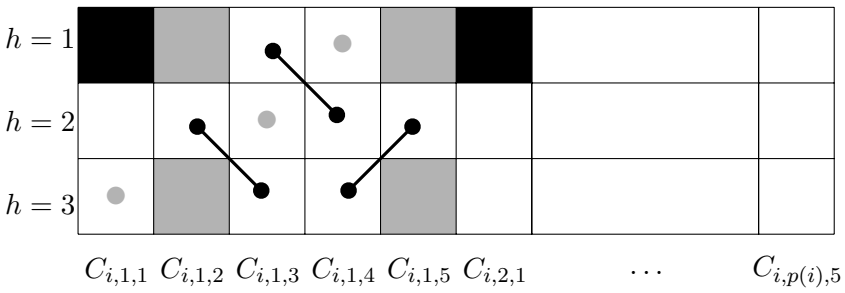
Now, once we know that our assignments for the teachers $T_{i,j,3/4}$ are consistent, we can use their scheduling assignments to encode the truth assignment for the variable $x_i$ into our schedule. The following schedule will correspond to $x_i = T$:



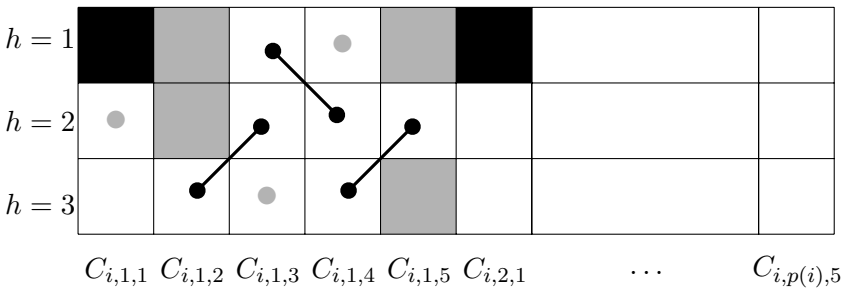

While the following schedule represents $x_i = F$:




In order to use these truth values to evaluate our clauses, we will make use of the classes $C_{i,j,2}$ and $C_{i,j,5}$. Note that each of these classes is taught in exactly one timeslot by the teachers currently at our disposal. If $x_i = T$, the times available for $C_{i,1,2}$ and $C_{i,1,5}$, for example would be the gray squares in the following timetables:




or




*The grayed-out circles represent the timeslots taken by our consistency-enforcing teachers $T_{i,p(i),c}$ and $T_{i,1,c}$ (the first two dots are for $T_{i,p(i),c}$, the third is for $T_{i,1,c}$). Note that these teachers are only necessarily consistent in the $h = 1$ timeslot: the other two timeslots can change from clause to clause, and are determined by our choice as to whether $C_{i,1,2}$ is taught in the timeslot $h = 2$ or $h = 3$.*

Note that we are forced to take the time slot $h = 2$ for the class $C_{i,1,5}$, but we have a choice for $C_{i,1,2}$. As we've said, this choice is independent across clauses.

If $x_i = F$, the situation is reversed: we can choose the timeslot taken for the $C_{i,j,5}$, but are forced to use the time slot $h = 2$ for the classes $C_{i,j,2}$.

We repeat this construction for all variables. As we've stated, each valid timetable corresponds to a truth assignment to $\phi$, and every truth assignment to $\phi$ can generate at least one timetable under our current restrictions.

We're almost done the reduction: we simply add one more set of teachers – one for each clause $D_j = (\ell_1 \vee \ell_2 \vee \ell_3)$ of $\phi$. We'll call these teachers $T_j$. Each $T_j$ will teach for three hours.

Now, if $\ell_1 = x_i$ for some $i$, and if $D_j$ is the $q^{th}$ clause that makes use of $x_i$, we assign $T_j$ to teach one hour of $C_{i,q,2}$. If $\ell_1 = \neg x_i$, we assign $T_j$ to teach one hour of $C_{i,q,5}$.

We assign $T_j$ to classes for $\ell_2$ and $\ell_3$ in the same fashion.

Once we are finished, the set $\mathcal{T}$ be br the set of teachers we have defined above, $\mathcal{C}$ to be the classes, $R$ to be the teacher/class assignments, and $H = \{1, 2, 3\}$, and return $\langle H, \mathcal{T}, \mathcal{C}, R\rangle$.

We argue that $\langle H, \mathcal{T}, \mathcal{C}, R\rangle \in$TIMETABLE-SCHEDULING iff $\phi \in$3SAT:

> Suppose $\phi \in$3SAT.
>
> Then it has a satisfying truth assignment $\tau$.
>
> As we have described above, we can encode this $\tau$ into an assignemt for the teachers $T_{i,j,3/4}$. This assignment will force the class assigned to the $T_{i,j,c}$ teachers for the first hour $h = 1$, and will also determine the class assignment for one of the two teachers $T_{i,j,2/3}$ or $T_{i,j,4/5}$ (i.e., if $x_i = T$, the teachers $T_{i,j,4/5}$ will all teach the class $C_{i,j,4}$ in the timeslot $h = 3$ and the class $C_{i,j,5}$ when $h = 2$. We would be able to choose the assignment for each of the $C_{i,j,2}$. If $x_i = F$, the situation is reversed.)
>
> Since $\tau$ is a satisfying truth assignment, at least one literal in every clause is satisfied. If, say, in clause $D_j$, a satisfying literal is $x_i$, then we can assign the teacher $T_j$ to teach the class $C_{i,q,2}$ in time $h = 2$. This teacher assignment forces the assignment of teachers $T_{i,q,2/3}$ and $T_{i,q-1,c}$, as we have seen above, but this assignment is possible. The teaching assignment for $T_{i,q-1,c}$ in class $T_{i,q,1}$ can be extended to a choice of the teachers $T_{i,q,1}$, $T'_{i,q,1}$, and $T''_{i,q,1}$ in the classes $C_{i,q,1}$, $C'_{i,q,1}$, and $C''_{i,q,1}$, as we have seen above.
>
> If the satisfing literal is $\neg x_i$, then $T_j$ can be assigned to the class $C_{i,q,5}$ when $h = 2$. The class assignments for $T_{i,q,4/5}$ and $T_{i,q,c}$ are then forced, and this assignment can be extended to $T_{i,q+1,1}$, $T'_{i,q+1,1}$, and $T''_{i,q+1,1}$ in the same fashion as in the previous case.
>
> The remaining classes taught by $T_j$ can be arranged as needed in time slots $h = 1$ and $h = 3$. As we have noted, the class assignments for different clauses in the same variable section of the timetable must be consistent as to their $T_{i,j,3/4}$ assignment, but can otherwise vary from clause to clause. The fact that $\tau$ is also consistent in its truth assignment from clause to clause ensures the consistency of the $T_{i,j,3/4}$ assignments. The remaining assignments can be extended to the $T_{i,q,2/3}$, $T_{i,q,4/5}$, $T_{i,q,c}$, $T_{i,q+1,1}$, $T'_{i,q+1,1}$, and $T''_{i,q+1,1}$ teachers, again following the argument above.
>
> This teaching assignment will cover all of the required teachers and classes:

We have ensured that ever teaching assignment is in a time when both the teacher and class are avaiable, and so property a) is satisfied.

We have arranged that every teacher uses every available time slot (and so teaches all required classes). So property b) is satisfied.

We have ensured that no two teachers are assigned the same class at the same time, so propert c) is satisfied.

We have only assigned one class to any one teacher at a time, and so property d) is satisfied.

So gives us a valid timetable $f$.

$\Rightarrow \langle H, \mathcal{T}, \mathcal{C}, R\rangle \in$TIMETABLE-SCHEDULING.

Suppose $\langle H, \mathcal{T}, \mathcal{C}, R\rangle \in$TIMETABLE-SCHEDULING.

Then there is a valid timetable $f$ satisfying properties a), b), c), and d).

As we have argued above, the teachers $T_{i,j,3/4}$ must be assigned consistently for any fixed $i$, and so we can use this assignment to read a truth assignment $\tau$: if the $T_{i,j,3/4}$ is assigned to the classes $C_{i,j,3}$ when $h = 1$, then $x_i = T$. Otherwise it is false.

Consider any clause $D_j$ in $\phi$. Since $f$ is a valid timetable, the teacher $T_j$ must teach some class in the timeslot $h = 2$.

If this class is $C_{i,q,2}$ for some $i$, then we know that $x_i$ is a literal in $D_j$. Moreover, if $T_j$ teaches $C_{i,q,2}$ when $h = 2$, then $T_{i,q,2/3}$ teaches $C_{i,q,2}$ when $h = 3$. We then know that $T_{i,q,2/3}$ teaches $C_{i,q,3}$ when $h = 2$, and so $T_{i,q,3/4}$ teaches $C_{i,q,3}$ when $h = 1$. So the variable $x_i$ is set to true, and therefore satisfies $D_j$.

Similarly, if $T_j$ teaches $C_{i,q,5}$ in timeslot $h = 2$ for some $i$, then $\neg x_i$ is a literal in $D_j$ and $x_i$ is set to false. Either way, $D_j$ is satisfied.

Since this is true for every clause $D_j$, $\tau$ satisfies every clause in $\phi$.

$\Rightarrow \phi \in$3SAT.

Finally, we can see that if $\phi$ has $n$ clauses and $k$ variables we have created $21n$ classes and $22n$ teachers. There are only three time slots avaiable, and we can determine the values of $\mathcal{T}$, $\mathcal{C}$, and $R$ in polynomial time. So this entire reduction is polytime.

So TIMETABLE-SCHEDULING is NP-complete, as required.

■

---

## 3.15 CUBIC-SUBGRAPH

Definition of CUBIC-SUBGRAPH (Problem from (Garey and Johnson, 1979). Reduction modified from (Stewart, 1994) – that reduction is from X1-3SAT, though):

**INPUT:** A graph $G = (V, E)$.

**QUESTION:** Does $G$ have a cubic subgraph (i.e., a set $V' \subseteq V$ such that the subgraph of $G$ containing all of the vertices in $V'$ and all of the edges between these vertices will be cubic)?

*A graph $G$ is cubic, or 3-regular, if every vertex has a degree of exactly 3. Also note that we don't accept empty graphs.*

**Reduced from:** 3SAT.

Firstly, we can see that this is in NP:

1) It is a yes/no problem.
2) Its certificate is the set $V'$ inducing the cubic subgraph (we can treat this as a list of vertices in $V$).
3) The size of this certificate is $\mathcal{O}(n)$, where $n$ is the number of vertices in $G$. This certificate is polynomial in the size of $G$.
4) A verifier for this certificate would be:

Let $\mathcal{V}$ ="On $\langle G = (V, E), V' \rangle$:"

1. Reject if any of the following checks fails:
   $\mathcal{O}(1)$ *time.*
2. Check that $V' \subseteq V$, and that no vertex ocurs more than once in it.
   $\mathcal{O}(n^2)$ *time.*
3. For all vertices $u \in V'$:
   $\mathcal{O}(n)$ *iterations.*
4. *count* := 0.
   $\mathcal{O}(1)$ *time per iteration.*
5. For $v \in V'$:
   $\mathcal{O}(n)$ *iterations.*
6. If $\{u, v\} \in E$, *count*++.
   $\mathcal{O}(n)$ *or* $\mathcal{O}(1)$ *time per iteration, depending on your implementation.*
7. Check that *count* == 3.
   $\mathcal{O}(1)$ *time per iteration.*
8. *Accept.*
   $\mathcal{O}(1)$ *time.*

5) We can see that this verifier runs in $\mathcal{O}(n^2)$ or $\mathcal{O}(n^3)$ time, and so is polytime in the size of $G$ .

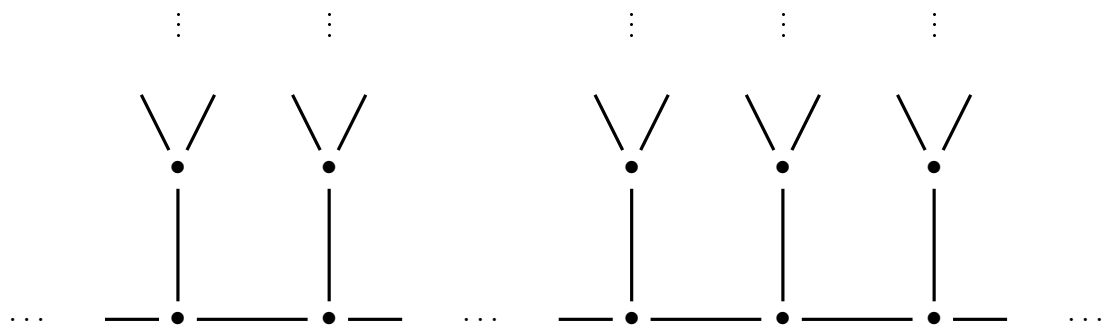
So CUBIC-SUBGRAPH is in NP.

**Reduction from 3SAT**:

Let an input 3CNF $\phi$ be given, where $\phi$ has $n$ clauses and $k$ variables. Note that we can assume using the lemma in section 2 that the clauses in $\phi$ each have three distinct variables.

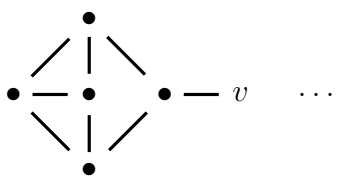
To start, let's have a look at some properties of cubic subgraphs. We'll also describe some useful widgets.

i) Firstly, consider what happens if we have any chain of the following pattern in $G$:



Each of these nodes has a degree of exactly three – so if any one of these vertices is removed, then its neighbours will have degree at most two, and so have to be removed as well (if we're building a cubic subgraph), and so their neighbours must also be removed.... We can see that either all of these nodes must be in a cubic subgraph, or none of them is.

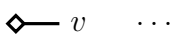
ii) Secondly, consider the following pattern:



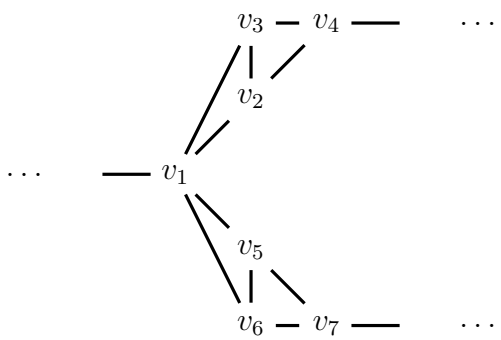

All of the vertices other than $v$ in this subgraph have degree three – so if we had a subgraph of $G$ that was almost cubic, but which had one degree two vertex $v$, we could attach this widget to $v$ to get a cubic subgraph. We could use this subgraph, for example, to cap off some or all of the undefined boundaries of the pattern in i).

You'll see, too, that if $v$ is not included in our subgraph, then none of the other vertices in this widget can be included, either.

We'll refer to this widget as a *tag*, and we'll denote it graphically as




iii) Finally, consider the following widget:



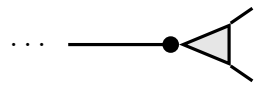
If we try to build a cubic subgraph using this widget, we see that $v_2$ is included, then so are $v_1$, $v_3$, and $v_4$. Similarly, if $v_5$ is included, then so are $v_1$, $v_6$, and $v_7$ (we can make similar arguments for $v_3$ and $v_6$).

But this means that we cannot include both $v_2$ and $v_5$ – otherwise, $v_1$ would have a degree of at least four.

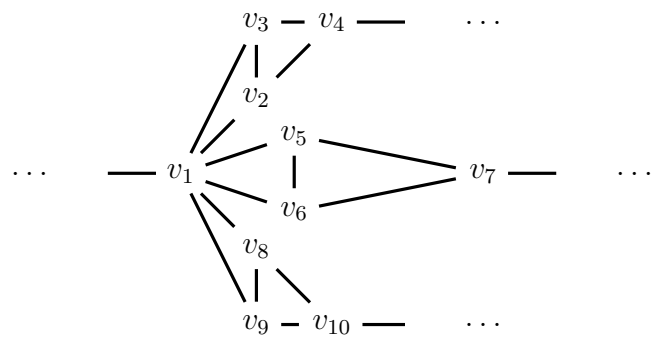
On the other hand, if $v_1$ is included, then it must have three neighbours. If these neighbours include the one on the left-hand boundary of the widget, we see that our cubic subgraph either contains $v_2$, $v_3$, and $v_4$, or it contains $v_5$, $v_6$, and $v_7$.

So either $v_1$ is not contained in our cubic subgraph (and in fact, none of these vertices is included), or $v_1$ is included, and this widget encodes a choice as to whether to extend our cubic subgraph through the $v_4$ or through the $v_7$ vertex.

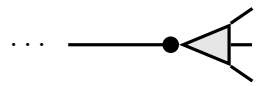
We'll denote this widget graphically as



In fact, we have as many branches as we want coming out of the widget:

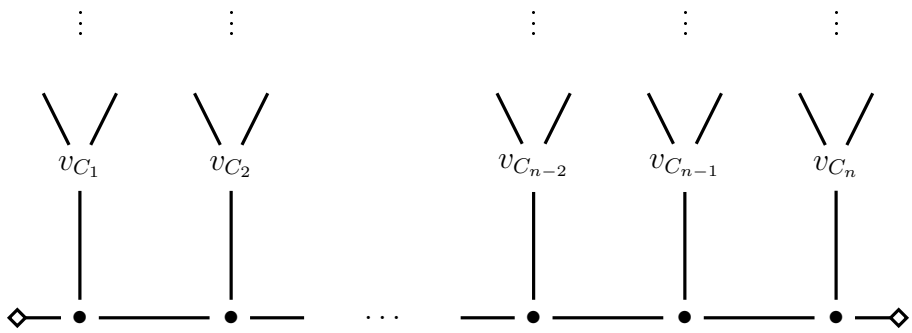



We'll denote multiple branch copies of this widget like so:



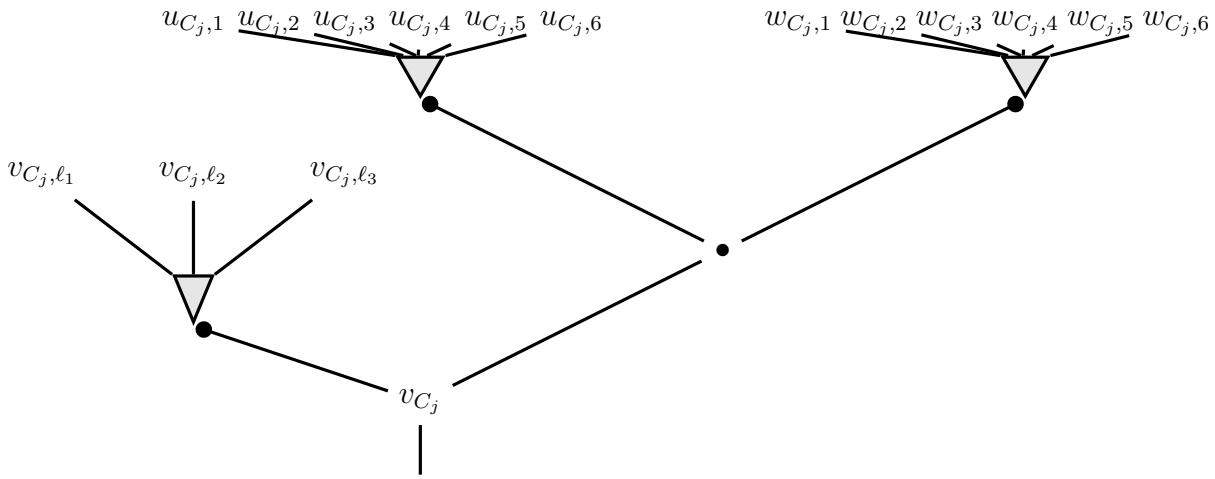
With these widgets, we are now ready to do our reduction.

If our input $\phi$ has $n$ clauses, we'll start by setting up the following widget, which we'll denote as $H_\phi$:



From our previous discussion, we know that any cubic subgraph of $G$ must either contain all of $H_\phi$ or none of it. We'll set things up so that any cubic subgraph of $G$ must contain all of $H_\phi$.

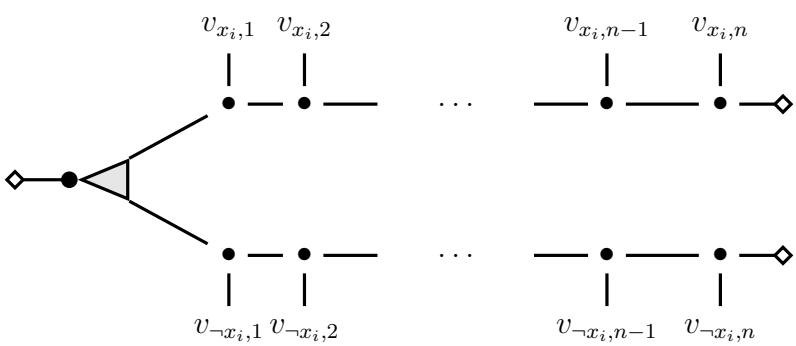
You can see that $H_\phi$ has $n$ nodes of the form $v_{C_j}$. These will be the base of our clause widgets. Each clause widget, which we will denote as $H_{C_j}$, will be of the form




From our discussion about the choice widgets, we know that if any node in these widgets in included in a cubic subgraph of $G$, then the node $v_{C_j}$ must be in that subgraph. Furthermore, if $v_{C_j}$ is included, then exactly one of the three $v_{C_j,\ell_a}$ branches in included, exactly one of the six $u_{C_j,b}$ branches in included, and exaclty one of the six $w_{C_j,b}$ branches is included. Since the $v_{C_j}$ vertices are common to both the $H_{phi}$ subgraph and the $H_{C_j}$ widgets, any cubic subgraph of $G$ either does not contain $H_\phi$ or any of the $H_{C_j}$ vertices, or it contains all of these widgets, and all of the $H_{C_j}$ widgets encode a choice as described above.

We will connect the nodes $v_{C_j,\ell_a}, a \in \{1,2,3\}$, to a set of variable widgets corresponding to the literals in $C_j$. The other nodes, the $u_{C_j,b}$ and the $w_{C_j,b}, b \in \{1,\ldots,6\}$, will correspond to any truth assignment of the other variables (they're basically buffer widgets so that the whole subgraph doesn't collapse). So let's look at the variable widgets.

For every variable $x_i, 1 \leqslant i \leqslant k$, we will create the following widget:



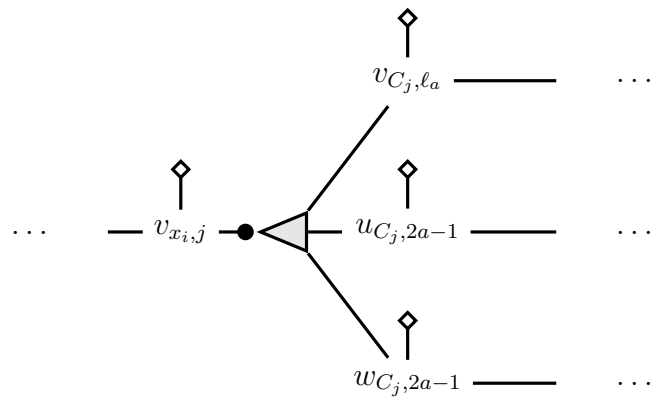

We can see that any cubic subgraph of $G$ either contains the entire upper branch (the one with the $v_{x_i,j}$ nodes) of this widget, the entire lower branch (the one with the $\neg v_{x_i,j}$

nodes), or none of these nodes. The upper branch encodes a choice of $x_i = T$, the lower branch encodes $x_i = F$.

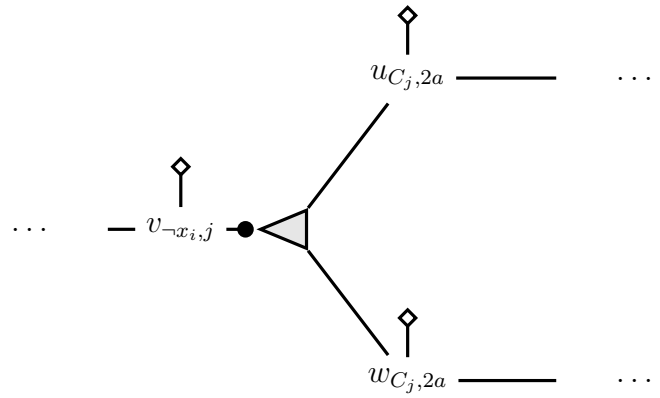
Note that there are $n$ nodes per variable widget of the form $v_{x_i,j}$, and $n$ nodes of the form $v_{\neg x_i,n}$. We'll attach these to the clause widgets described above.

Given a clause $C_j = (\ell_1 \vee \ell_2 \vee \ell_3)$, then:

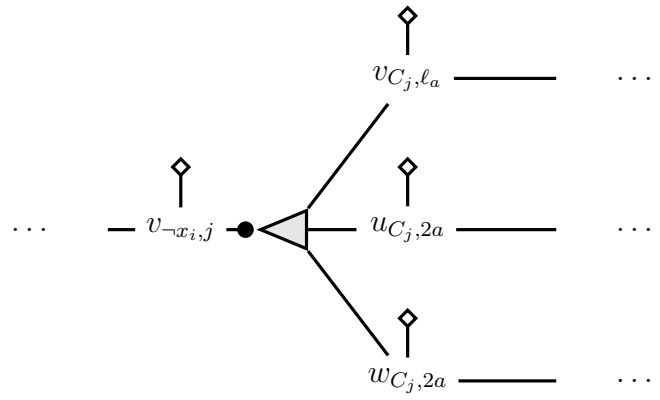
- If $x_i = \ell_a, a \in \{1, 2, 3\}$, we attach the following widget to $v_{x_i,j}$:



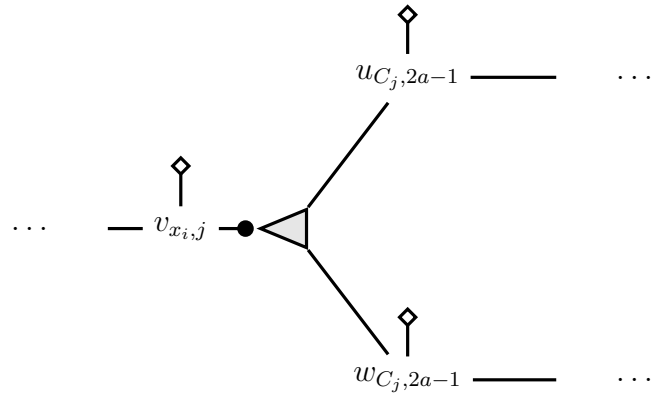

  Likewise, we attach the following widget to $v_{\neg x_i,j}$:




- If $\neg x_i = \ell_a, a \in \{1, 2, 3\}$, we attach the following widget to $v_{\neg x_i,j}$:



Likewise, we attach the following widget to $v_{\neg x_i,j}$:




- If neither $x_i$ nor $\neg x_i$ occurs in $C_j$, we replace both $v_{x_i,j}$ and $v_{\neg x_i,j}$ with tag widgets.

Once we've done this for every variable and clause, we return the resulting $\langle G \rangle$.

We argue that $\langle \phi \rangle \in$3SAT iff $\langle G \rangle \in$CUBIC-SUBGRAPH.

**Proof that $\langle \phi \rangle \in$3SAT iff $\langle G \rangle \in$CUBIC-SUBGRAPH:**

**Suppose that $\langle \phi \rangle \in$3SAT:**

Then there is some truth assignment $\tau$ to $\phi$ such that every clause $c_k$ in $\phi$ has at least one true literal and one false literal under $\tau$.

For every variable $x_i$ in $\phi$, we can start a cubic subgraph by choosing the appropriate branch of the variable widget – we include the upper branch if if $x_i$ is true, and the lower if $x_i$ is false.

We also include the $H_{phi}$ widget, as well as the $H_{C_j}$ widgets for every clause $C_i$.

We We see from our discussions above that each of these subgraphs is cubic, except for the connections beteween the variable and the clause widgets, which we have not yet specified.

Now, for any clause $C_j = (\ell_1, \ell_2, \ell_3)$, we know that at leat one of $\ell_1, \ell_2$, or $\ell_3$ is true under $\tau$. So one of the $\ell_a$ is either $x_i$ for some true $x_i$ or $\neg x_i$ for some false $x_i i$. Suppose that the satisfied variable is $\ell_a = x_i$ (the false case is similar). Since we've chosen the true branch of the $x_i$ widget to be part of our cubic subgraph, the $v_{x_i,j}$ node willl be part of our subgraph. For the branch at this point, we choose the path leading to the $v_{C_j,\ell_a}$ node of $H_{C_j}$.

There are two more literals in the clause $C_j$, and these literals may or may not be satisfied. You can see that there are also two more braches of the $H_{C_j}$ to be filled. We connect these to the variable widgets for the remaining two literals in $C_j$.

*Notice that each of the other two variables can connect to the $u_{C_j,b}$ and $w_{C_j,b}$ nodes, whether they satisfy $C_j$ or not – we only require one satisfying literal. Also note that each node can only connect to one of these two branches at a time.*

If we do this for every clause and literal, there will be no vertex in our subgraph not of degree three, and so we'll have a cubic subgraph: $\langle G \rangle \in$CUBIC-SUBGRAPH.

**Suppose that $\langle G \rangle \in$CUBIC-SUBGRAPH:**

Then it has a cubic subgraph $G'$.

We argue that $G'$ must contain $H_\phi$:

> Recall that $G'$ either contains all of $H_\phi$ or none of it. If it contains none of it, then it contains none of the $v_{C_j}$ nodes, either.
>
> But then $G'$ can contain no node in any branch of the $H_{C_j}$ widgets. In particular, it contains none of the $v_{C_j,\ell_a}$ nodes, nor does it contain any $u_{C_j,b}$ or $w_{C_j,b}$ nodes.
>
> But then there are node missing from the variable widget of any $x_i$ connecting to any clause (i.e., all of the variable widgets). In fact, both the true and false branches of this widget will be missing all of the output nodes from the clause sections of the widget.
>
> This means that the entire clause sections will be missing from these widgets, and so each variable will be missing vertices from both branches of the base sections of their widgets.
>
> Therefore, no vertex from any variable widget can be included.
>
> Since every vertex in $G$ is one of these types of vertices, $G'$ would have to be empty. But we don't allow an empty cubic subgraph, and so we have a contradiction.
>
> So the entire $H_\phi$ subgraph is in $G'$.

Since the entire $H_\phi$ subgraph is in $G'$, each of the $v_{C_j}$ vertices is included, too.

But then, each $H_{C_j}$ widget is included, as well (up to some choice of branches). So one of the $v_{C_j,\ell_a}$ nodes is in $G'$.

We can read off the $\ell_a$ to find a variable assignment that satisfies $C_j$. Furthermore, we can find such an assignment for every clause in $\phi$.

*Note that we can only include one of the true or false branches of any variable widget, and so this variable assignment will be consistent.*

If we put all of these variable assignments together, we'll get a partial truth assignment that satisfies every clause of $\phi$. If any variables are not fixed in this assignment, we can assign them as we please to get a full truth assignment $\tau$ that satisfies $\phi$.

So $\phi$ has a satisfying truth assignment $\tau$, and so $\langle \phi \rangle \in$3SAT, as required.

Finally, we see that if $\phi$ has $r$ variables and $n$ clauses, then:

- $G$ has one $H_\phi$ widget, which has $2n + 10$ vertices.
- $G$ has $n$ clause widgets, each of which has 49 vertices (not counting the connecting vertices). So this gives us $49n$ vertices in total.

- $G$ has $k$ variable widgets, each of which has $2n + 17$ nodes in the basic widget and at most 17 nodes per clause, giving a total of no more that $19kn + 17k$ nodes.

So all told, $G$ will have no more than $19kn + 51n + 17k + 10$ nodes.

We can tell in polytime whether any pair of these nodes has an edge, and so this whole construction is polytime, as required.
So 3SAT $\leqslant_p$ CUBIC-SUBGRAPH, and so CUBIC-SUBGRAPH is NP-complete.

■

---

## 3.16 NONOGRAM

Definition of NONOGRAM (Problem and reduction from (Ueda and Nagao, 1996).):

**Input:** Two numbers $h, w \in \mathbb{N}$, and two sequences $R = \langle \overline{r_1}, \ldots, \overline{r_h} \rangle$ and $C = \langle \overline{c_1}, \ldots, \overline{c_w} \rangle$ of lists of non-negative integers.

*Note that each $\overline{r_i}$ and $\overline{c_j}$ is itself a series of non-negative integers. That's why we're denoting them as verctors. The lengths of these sequences may vary, though.*

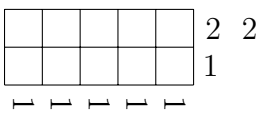
**Question:** Is there an $h \times w$ $\{0, 1\}$ matrix $P$ that satisfies the following properties:?

- For every row $P_{i,\cdot}$ of $P$, the lengths of the maximal contiguous blocks of 1s match the numbers in $\overline{r_i}$.
- For every column $C_{\cdot,j}$ of $P$, the lengths of the maximal contiguous blocks of 1s match the numbers in $\overline{c_j}$.

Let's look at a simple instance of this problem as an example. Suppose we have the following problem instance:

$h = 5, w = 2.$
$R = \langle (2, 2), (1) \rangle$
$C = \langle (1), (1), (1), (1), (1) \rangle$

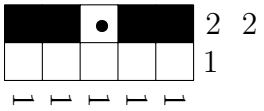
This corresponds to the puzzle



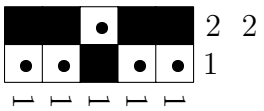

Now, this puzzle is asking us to fill in the top row of the table using two blocks of two ones, and the bottom row with one blaock of a single one. The columns should each have a sincegle filled block.

We'll indicate a filled block (a 1 in $P$) as a filled black square. If there's a block that we know is a 0 in $P$, we'll indicate it with a dot ("•"). If we don't know if a square is 0 or 1 it'll be left blank.

Now, in out example puzzle we see that the top row needs two blocks of contiguous ones. There are five spaces, and the two blocks must be seaparate. So they have to be separated by a space in the middle. The only way to do this is to fill it in this way:




Now, each column has exactly one block of one, so the leftmost two columns and the rightmost two columns must have a zero on the bottom, while middle must have a one:




And at this point we've filled the table, and so we've solved the puzzle. So this is a *yes*-instance of NONOGRAM. If there were no way of finishing the puzzle, it would be a *no*-instance.

**Reduced from:** 3DM.

*Note: This is a logic puzzle that can show up under several names including LOGIC ART and PICROSS.*

Proof that NONOGRAM is in NP:

1) It is a yes/no problem.

2) A certificate would be the $\{0,1\}$ matrix $P$.

3) Since $|R| = h$ and $|C| = w$, and since $P$ is an $h \times w$ $\{0,1\}$ matrix, this certificate is polynomial in the size of $\langle R, C\rangle$.

4) A verifer $V$ for this language would be:

   $\mathcal{V}$ = "On input $\langle h, w, R, C, P\rangle$:

   1. Check that $P$ is an $h \times w$ $\{0,1\}$ matrix.

      $\mathcal{O}(hw)$ *time.*

   2. For $i = 1$ to $h$:

      $\mathcal{O}(h)$ *iterations.*

   3. Loop over $P_{i,\cdot}$ and find the sizes $s_i$ of the contiguous blocks of 1s.

      $\mathcal{O}(w)$ *time after amortization.*

   4. Check that $s_i == r_i$, and *reject* if they are different.

      $\mathcal{O}(w)$ *time per iteration.*

   5. For $j = 1$ to $w$:

      $\mathcal{O}(w)$ *iterations.*

   6. Loop over $P_{\cdot,j}$ and find the sizes $s'_j$ of the contiguous blocks of 1s.

      $\mathcal{O}(h)$ *time after amortization.*

   7. Check that $s'_j == c_j$, and *reject* if they are different.

      $\mathcal{O}(h)$ *time per iteration.*

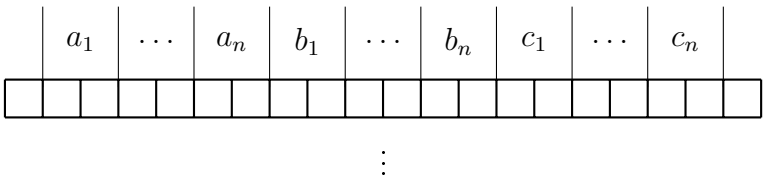
   8. *Accept.*

5) We can see that this verifier runs in $\mathcal{O}(hw)$ time, and so is polytime in the size of $\langle h, w, R, C\rangle$ *(again, recall that $|R| = h$ and $|C| = w$).*

Proof that 3DM $\leqslant_p$ NONOGRAM:

Let an instance $\langle A, B, C, T\rangle$ of 3DM be given. We'll construct an instance of NONOGRAM as follows:

- The columns of the nonogram puzzle will roughly correspond to the elements of $A \cup B \cup C$: There will be a set of columns corresponding to the elements of $A$, then a set of columns for $B$, and finally a set for $C$:




- The rows will roughly correspond to the triples in $T$. Each row will be made up of four "bars" of 1s: One at the left hand side of the table, one separating the $A$ and the $B$ element, one separating the $B$ and the $C$ elements, and one on the right hand side of the table.

  As an example, suppose we have the 3DM instance

  $$A = \{a_1, a_2, a_3\}, B = \{b_1, b_2, b_3\}, C = \{c_1, c_2, c_3\}$$

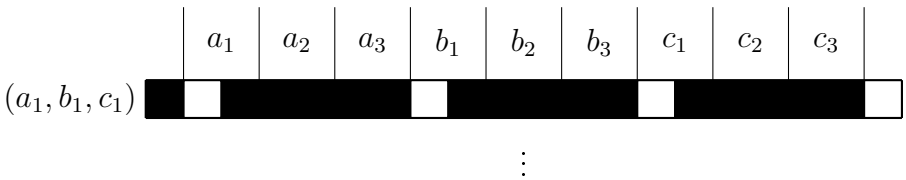
  with the triples

  $$T = \{(a_1, b_1, c_1), (a_1, b_2, c_3), (a_2, b_2, c_3), (a_2, b_3, c_2), (a_3, b_2, c_3), (a_3, b_3, c_2)\}.$$

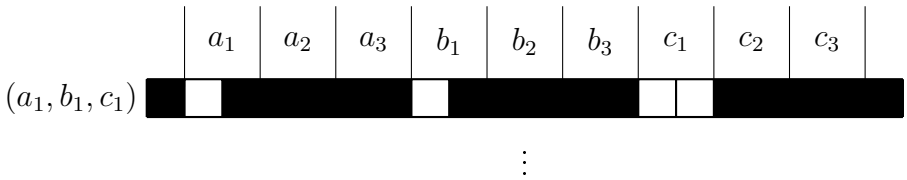
  *Note that this is also the example used in (Ueda and Nagao, 1996).*

  Then, the first row of our puzzle would correspond to the triple $(a_1, b_1, c_1)$. This row would take the numbers $(1, 5, 5, 5)$ (we'll see why we use those numbers in a minute).

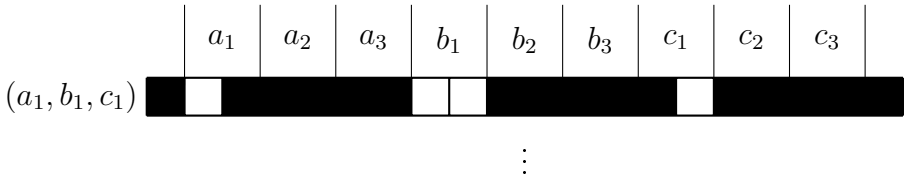
  If we do this, there are five ways of filling out the top row:



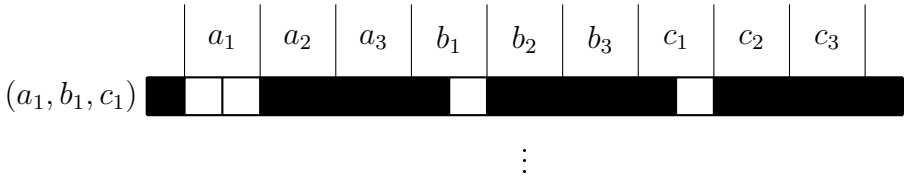

  or



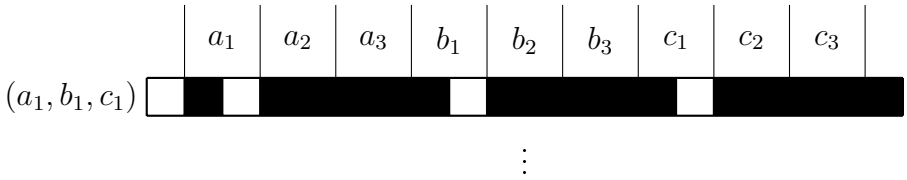

or




or




or




We'll use the numbers on the columns to ensure that only the second or the fourth conficguration is ever possible.

Furthermore, we'll set up our numbers so that the second configuration (the bars shifted to the left) corresponds to a choice of $(a_1, b_1, c_1) \in M$, while the fourth configuration (the bars shifted to the right) corresponds to a choice of $(a_1, b_1, c_1) \notin M$.

*You'll notice we've used two columns each for the elements in $A \cup B \cup C$. We've done this so we have just enough room for these configurations, but not enough room for anything else. We've also added two columns on each side. You'll see soon that they act as anchors for our blocks: we'll use the numbers $\overline{c_1}$ and $\overline{c_{6n+2}}$ for these columns to enusre that neither the left-hand block nor the right-hand block can move from the ends of the matrix.*

You can now see where the numbers $(1, 5, 5, 5)$ come from: If we're looking at a triple $(a_i, b_j, c_k)$ (in this example $(a_1, b_1, c_1)$), then:

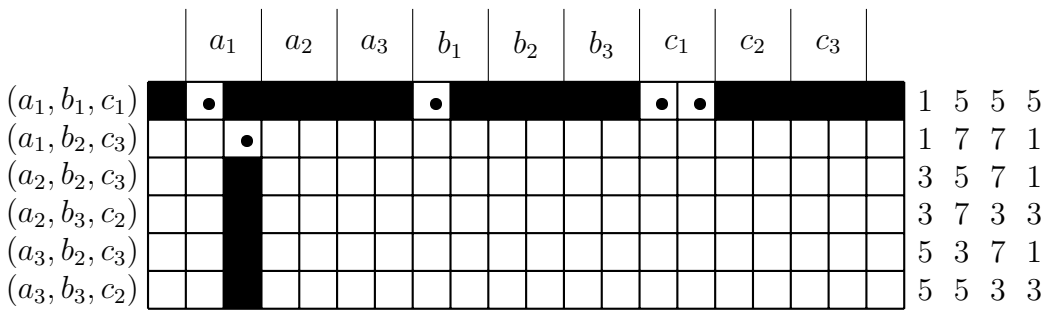
- The first number should encode a block that extends from the left of the matrix to the start of $a_i$ ($a_1$ in this example). So the length of the first block is $1 + 2(i-1) = 2i - 1$ (in our example, this gives $2 \times 1 - 1 = 1$).
- The second number should fill in every column between $a_i$ and $b_j$, and add one extra square so it intersects exactly one of the $a_i$ or $b_j$ columns. There are $n$ elements in each of $A$, $B$, and $C$, and so the number of elements between $a_i$ and $b_j$ is $n + j - i$.

  So in total the second block is of length $2(n+(j-1)-i)+1 = 2(n+j-i)-1$. In our example this gives $2(3+1-1)-1 = 5$.
- Similarly, the third number should fill in every column between $b_j$ and $c_k$, and add one extra square so it intersects exactly one of the $b_j$ or $c_k$ columns. Using a similar argument to the pprevious one, the number of elements between $b_j$ and $c_k$ is $n + k - j$.

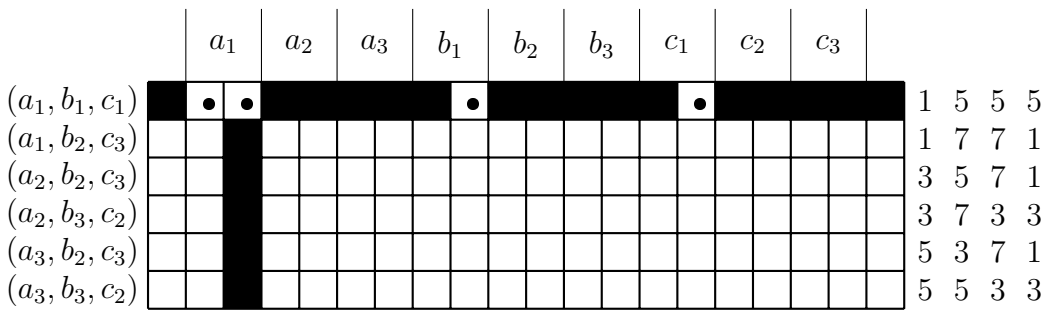
  So in total the third block is of length $2(n+(k-1)-j)+1 = 2(n+k-j)-1$. In our example this gives $2(3+1-1)-1 = 5$.
- The final number should encode a block from the end of the $c_k$ columns to the right side of the matrix. So the number here is $2(n-k)+1$. In our example this gives $2(3-1)+1 = 5$.

You can see that the numbers for the other triples are: $(1,7,7,1), (3,5,7,1), (3,7,3,3), (5,3,7,1)$, and $(5,5,3,3)$.

Now, we can't just stack the rows corresponding to the triples on top of each other: If we did, we'd end up having to encode the value of $M$ into the $C$ vectors. For example, you can see that in our example instance, the element $a_1$ occurs in two triples: $(a_1, b_1, c_1)$ and $(a_1, b_2, c_3)$. If we use only one row for each triple, the top row and third column would end up being either:



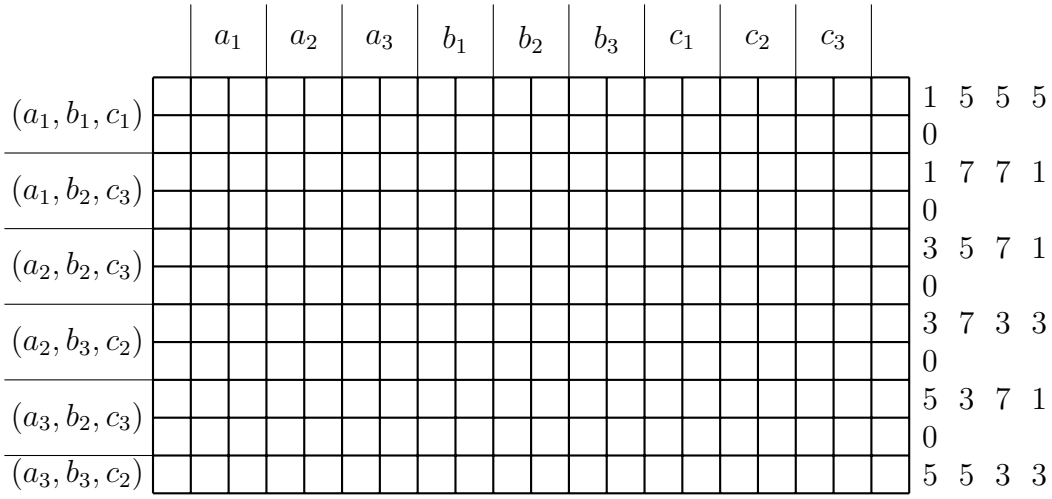

or



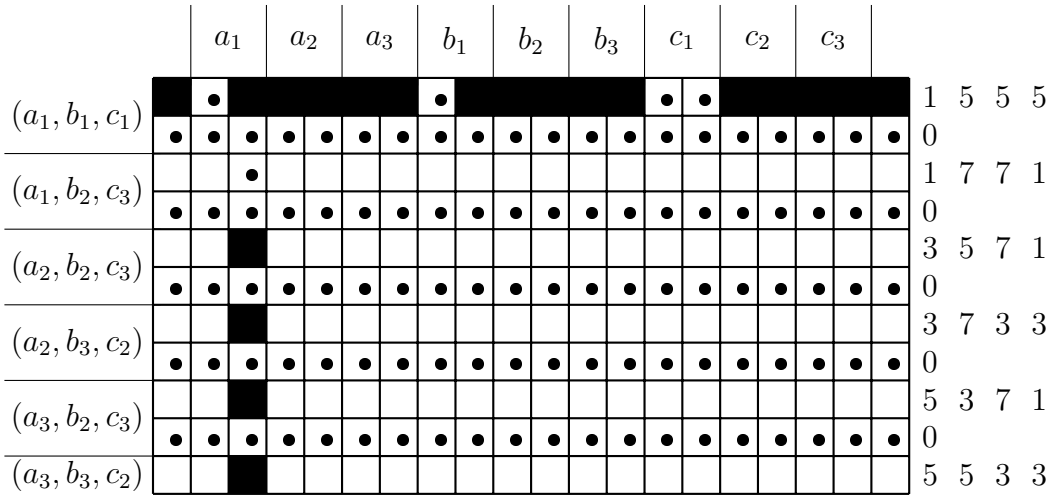
*Remember that we intend to use the column numbers to force this behaviour!*

If we did this, the numbers in $\overline{c_3}$ would either be $(1, 4)$ or $(5)$, and which one we chose would completely determine whether we could choose $(a_1, b_1, c_1)$ or $(a_1, b_2, c_3)$ as out element in $M$. So we need to fix this behaviour.

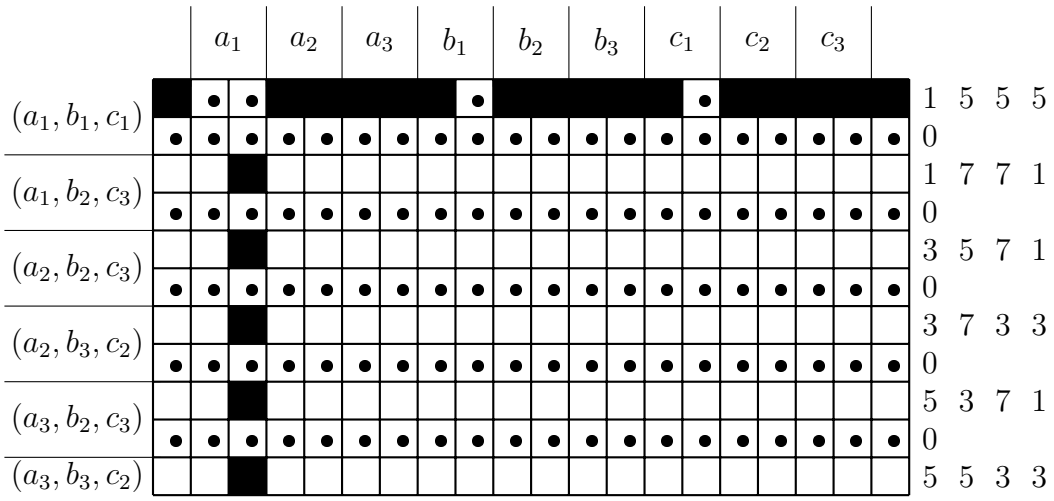
The way to do this is to put a one-line break between every triple in our puzzle:




Now, the choices we get for the top row and third column are:

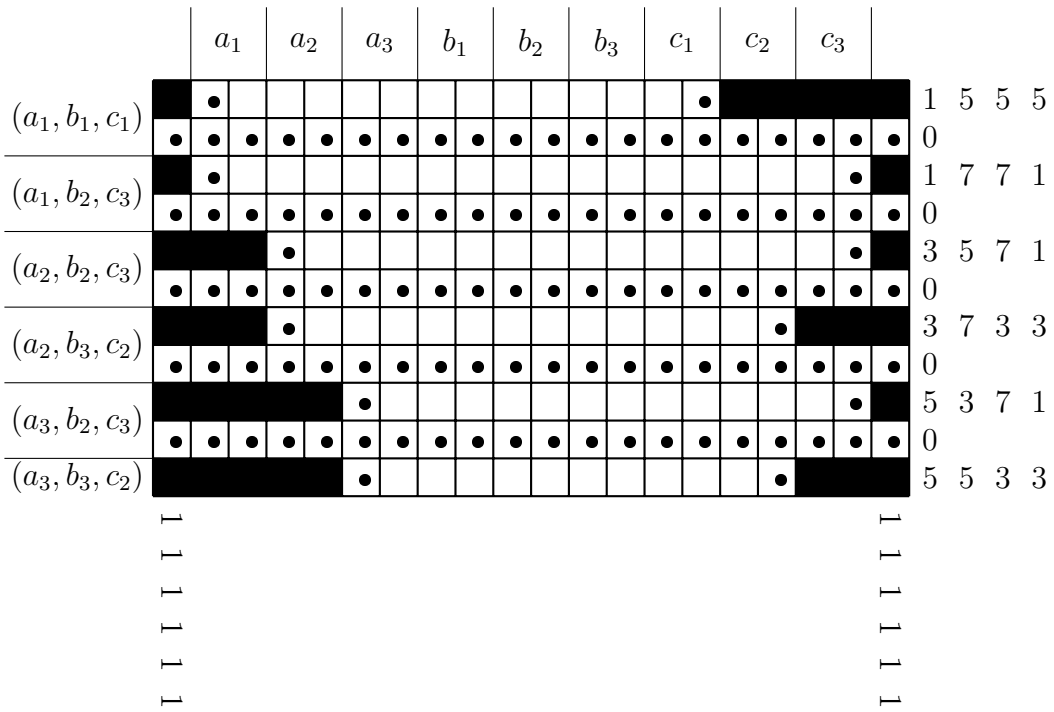



or




And in either case we get a $\overline{c_3}$ of $(1, 1, 1, 1, 1)$. So we don't accidentally force a choice of $M$.

Of course, we've said that we want to force only one of these two possibilities to occur. The first thing we can do to enforce this is to disallow the far-left and the far-right blocks from moving. We can do this by making the lefmost and rightmost column numbers $\overline{c_1}$ and $\overline{c_{20}}$ both be set to $(1, 1, 1, 1, 1, 1)$ (a single 1 for each triple in $T$). Now, there are six triples in our examples, and six 1s in the leftmost and rightmost columns, and the only rows that can contain 1s are the rows corresponding to the triples. So we have to fill out the ends of our puzzle like so:




To fill in the rest of the $\overline{c_i}$, we'll define a couple of variables.

- Let $t = |t|$ (so $\overline{c_1} = \overline{c_{20}} = (\overbrace{1\ \ 1\ \ \ldots\ \ 1}^{t\,\text{times}})$ in our example).
- For $v \in A \cup B\ cupC$, let $N(v)$ be the number of triples in $T$ that contain $v$. So in our example, $N(a_1 = 2)$ and $(N(c_3)) = 3$.

Now, we want every $a_i$ to be in exactly one triple in $M$, and we've said that this corresponds to the choice of pushing all of the bars on a row to the left.

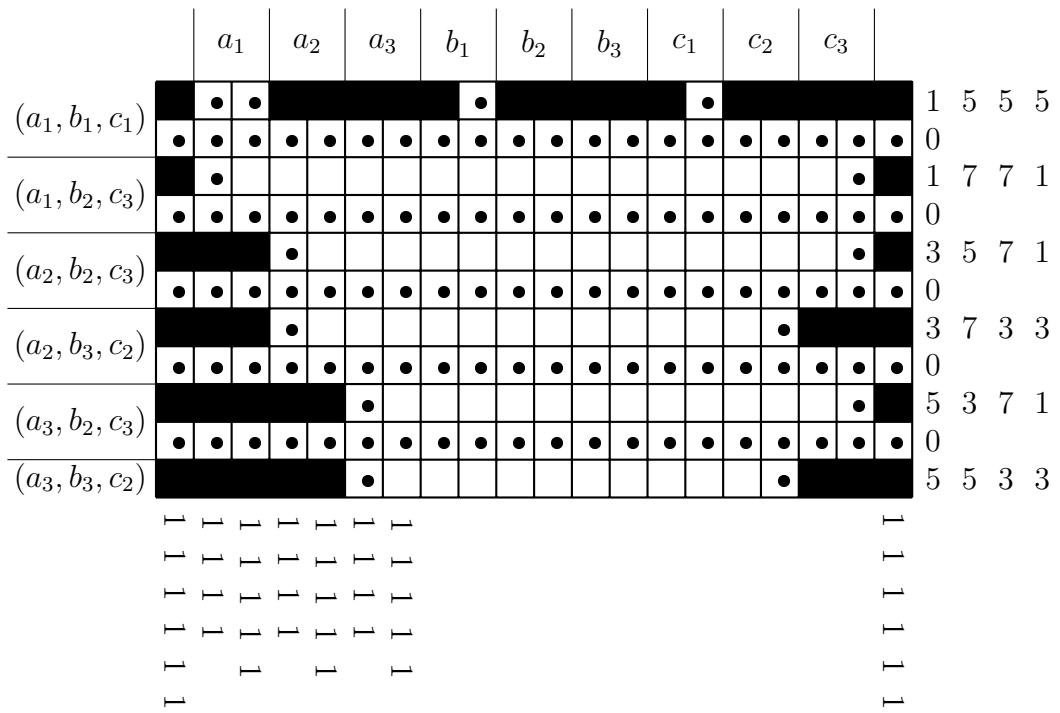
So suppose that, for some triple $(a_i, b_j, c_k)$, the bars will be pushed to the left. Then on the corresponding row the first of the $a_i$ columns will be 0 (since this is where the break after the leftmost row block is), but the second $a_i$ column will be set to 1. Remember that only one row per $a_i$ will satisfy this property.

On the other hand, if the bars are pushed to the right, both $a_i$ columns will be set to 0.

Finally, the $a_i$ columns for every non-zero row corresponding to a triple not containing $a_i$ will both be 1.

So the value given by $C$ for the first $a_i$ column will be $\overline{c_{2i}} = (\overbrace{1, 1, \ldots, 1}^{n-N(a_i\ \text{times}})$ (i.e., 1 repeated $n - N(a_i)$ times). The second column will have the numbers $\overline{c_{2i+1}} = (\overbrace{1, 1, \ldots, 1}^{n-N(a_i)+1\ \text{times}})$. In our example this will give us the values $\overline{c_2} = (1, 1, 1, 1)$, $\overline{c_3} = (1, 1, 1, 1, 1)$, $\overline{c_4} = (1, 1, 1, 1)$, $\overline{c_5} = (1, 1, 1, 1, 1)$, $\overline{c_6} = (1, 1, 1, 1)$, and $\overline{c_7} = (1, 1, 1, 1, 1)$.

*Note: there are $n$ elements in $A$, and each one has exactly one row* not *pushed to the right, and so the total number of rows pushed to the right is $t - n$. These rows can only be completed in one way, given the constraints we've already given: If we push the first row to the right by setting both $a_1$ columns to 0, we'd have to fill in the whole row as*



*If we chose to set the bars to the left we'd still have a couple of choices. Regardless of the choices we make, though, the current constraints will require $n$ such rows.*

Similarly, we want every $c_k$ to be in exactly one triple in $M$.

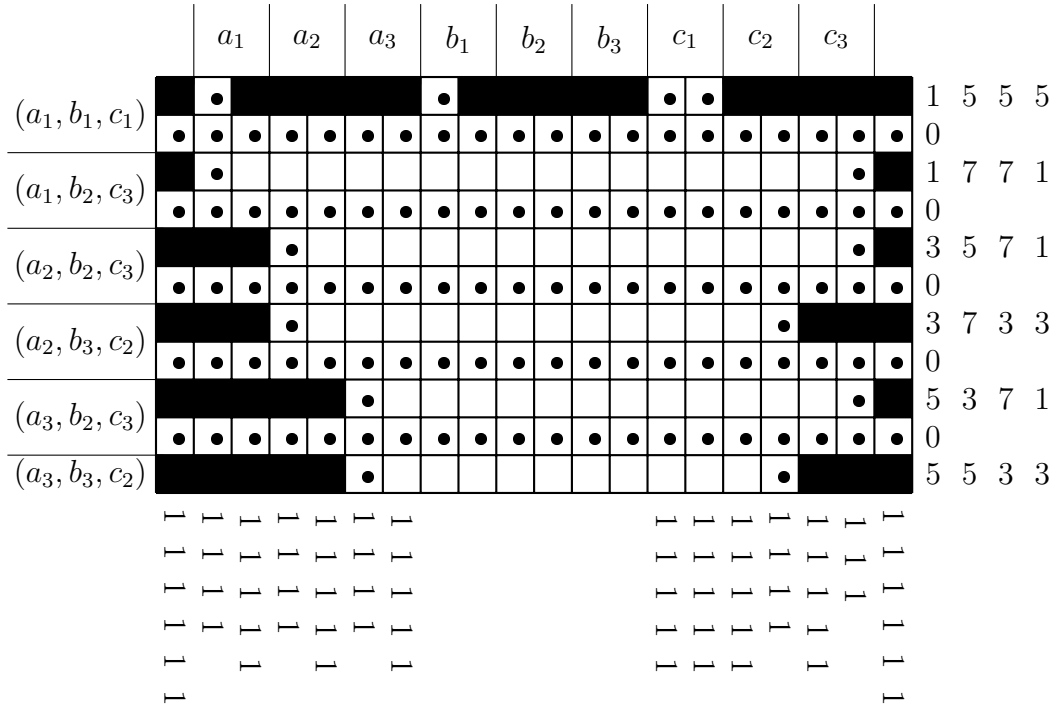
Suppose that, for some triple $(a_i, b_j, c_k)$, the bars will be pushed to the right. Then on the corresponding row the first of the $a_i$ columns will be 1, but the second $c_k$ column will be set to 0 (since this is where the break before the rightmost row block is).

On the other hand, if the bars are pushed to the left, both $c_k$ columns will be set to 0. Remember that only one row per $c_k$ will satisfy this property.

Finally, the $c_k$ columns for every non-zero row corresponding to a triple not containing $c_k$ will both be 1.

So the value given by $C$ for the first $c_k$ column will be $\overline{c_{4n+2k}} = (\overbrace{1, 1, \ldots, 1}^{n-1 \text{ times}})$. The second column will have the numbers $\overline{c_{4n+2k+1}} = (\overbrace{1, 1, \ldots, 1}^{n-N(c_k \text{ times}})$. In our example this will give us the values $\overline{c_{14}} = (1, 1, 1, 1, 1)$, $\overline{c_{15}} = (1, 1, 1, 1, 1)$, $\overline{c_{16}} = (1, 1, 1, 1, 1)$, $\overline{c_{17}} = (1, 1, 1, 1)$, $\overline{c_{18}} = (1, 1, 1, 1, 1)$, and $\overline{c_{19}} = (1, 1, 1)$.

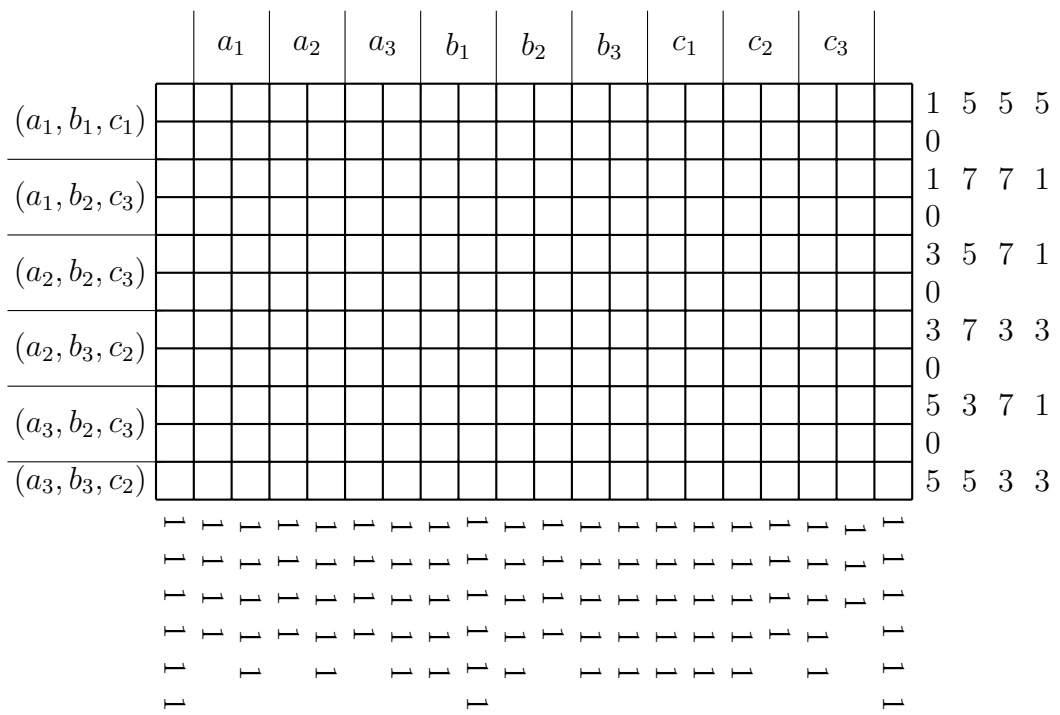
*Note: there are also $n$ elements in $C$, and each one has exactly one row pushed to the left, and so the total number of rows pushed to the left is $n$. These rows can only be completed in one way, given the constraints we've already given: If we push the first row to the left by setting both $c_1$ columns to 0, we'd have to fill in the whole row as*



So we see that the current constriants on pur puzzle will force us to choose $n$ rows to be pushed to the left, and $t - n$ rows to be pushed to the right, and that every element of each of these rows is determined by this choice. Furthermore, each $a_i$ can occur in exactly one of our left choices, and each $c_k$ occurs in exactly one of our left choices as well. So every way of completing this puzzle corresponds to a size-$n$ subset $M \subseteq T$ in which each element of $A$ and each element of $C$ occurs exactly once. We just need to ensure that every element of $B$ occurs ecactly once in our choice as well.

Now, for any $b_j$ and triple $(a_i, b_j, c_k)$, we know that choosing to include $(a_i, b_j, c_k)$ in $M$ corresponds to bushing its bars to the left. This will leave the leftmost column of $b_j$ as a 0 and the rightmost column as a 1. A choice not to include $(a_i, b_j, c_k)$ in $M$ will likewise leave the leftmost column of $b_j$ as a 1 and the rightmost column of $b_j$ as a 0.

We want to choose exactly one triple with $b_j$ in it, so if we set the left $b_j$ column to $\overline{c_{2n+2j}} = (\overbrace{1, 1, \ldots, 1}^{n-1 \text{ times}})$ and the right $b_j$ column to be $\overline{c_{4n+2k+1}} = (\overbrace{1, 1, \ldots, 1}^{n-N(b_j)+1 \text{ times}})$ we'll get this result. So the final puzzle in our example would be




With this puzzle, every solution will correspond to a choice of $n$ rows to be set to the left, and these rows will correspond to a set of triples. By our choices of the numbers in $C$, we see that every $a_i$, $b_j$, and $c_k$ occurs exactly once in the triples set to the left. So every solution to our NONOGRAM problem corresponds to a solution $M \subseteq T$ to our 3DM instance.

Similarly, if $M \subseteq T$ is a solution to our 3DM instance, then we can set the rows of $P$ corresponding to the triples in $M$ to the left, and the others to the right.

We can see that each $a_i$, $b_j$, and $c_k$ will occur in exactly one triple, and from our arguments above this means that the columns numbers will be satisfied. So this 3DM solution gives us a solution to our puzzle. This means that the solutions for our NONOGRAM puzzle correspond to solutions to our 3DM instance.

Moreover, we can see that different solutions to our 3DM will correspond to a different choice of rows in our puzzle to be pushed left (i.e., to different puzzle solutions), and vice-versa.

In fact, there is exactly one way to fill in the values of $P$ for every certificate for our original 3DM isntance. If we accept two certificates as equivalent if they encode the same matrix $P$, then this is a parsimonious reduction.

We set $h = 2t$ (the number of rows in our table) and $w = 2n+2$ (the number of columns), and $R$ and $C$ to be the row and column numbers. From our discussion we can see that $\langle A, B, C, T\rangle \in$3DM iff $h, w, \langle R, C\rangle \in$NONOGRAM.

Finally, we can see that our puzzle will contain $w = 2n + 2$ columns and $h = 2t - 1$ rows. So $|R| = 2t - 1$ and $|C| = 2n + 2$. Each $\overline{c_i} \in C$ will contain numbers that sum up to at most $h$, and so will certainly take up $\mathcal{O}(h)$ space. Similarly, Each $\overline{r_i} \in R$ will contain numbers that sum up to at most $w$, and so will certainly take up $\mathcal{O}(w)$ space. Determining the values of these numbers takes polynomial time per number, and so the entire reduction is polytime, as required.

So NONOGRAM is NP-complete.

■

# References


Shimon Even, Alon Itai, and Adi Shamir. On the complexity of time table and multi-commodity flow problems. In *16th annual symposium on foundations of computer science (sfcs 1975)*, pages 184–193. IEEE, 1975.

Michael R Garey and David S Johnson. *Computers and intractability: A guide to the theory of NP-completeness*. WH Freeman, 1979.

Iain A Stewart. Deciding whether a planar graph has a cubic subgraph is np-complete. *Discrete Mathematics*, 126(1-3):349–357, 1994.

Nobuhisa Ueda and Tadaaki Nagao. Np-completeness results for nonogram via parsimonious reductions. *preprint*, 1996.